

Importante

La descripción comentario dentro del en la creación de tablas es porque se me menciona que eran necesarias explicaciones a fondo sobre los esquemas. Personalmente, quitaría todo ese comentario incluso del `.md / .pdf`.

- Consideraré práctico minimizar capturas de pantalla.
 - Los bloques de código no caben en pantalla.
1. Mi solución enumera cada bloque de código con `-- [0PXXX]`, y este número coincide con el `Action Output`.
 2. Las capturas de código las he pasado ha escrito en el documento.
En la práctica hacen falta solo capturas en cada resultado en `Results Grid`: he sido meticuloso en cuanto a capturar todos los `Action Output` al menos una vez.

Lvl. 1

Ex. 1

A partir dels documents adjunts `estructura_dades` i `dades_introduir` importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Con tal de facilitar una ejecución sin interrupciones, el código en este ejercicio es una copia exacta del archivo `N1-Ex.1__estructura_dades.sql`. También dobla como documentación del esquema, ya que se pide explicarlo.

Creamos la base de datos:

```
-- [0P001]
CREATE DATABASE IF NOT EXISTS transactions;
```

```
-- [0P002]
USE transactions;
```

Creamos la tabla `company`:

```
-- [0P003]
CREATE TABLE IF NOT EXISTS company (
```

```

-- Identificador unico de cada fila en la tabla
id VARCHAR(15) PRIMARY KEY,
-- Nombre de la empresa
company_name VARCHAR(255),
-- Numero de telefono de la empresa
phone VARCHAR(15),
-- Email de la empresa
email VARCHAR(100),
-- Pais donde opera la empresa
country VARCHAR(100),
-- Pagina web de la empresa
website VARCHAR(255)
);

```

Creamos la tabla transaction:

```

-- [OP004]
CREATE TABLE IF NOT EXISTS transaction (
-- Identificador unico de cada fila en la tabla
id VARCHAR(255) PRIMARY KEY,
-- Referencia a identificadores unicos
-- de la tabla "credit_card" que aun no hemos creado.
-- Cada fila de la tabla "transaction" esta ligada
-- a 0 o 1 filas de "credit_card", ya que
-- la variable es nutable. [2]
credit_card_id VARCHAR(15) REFERENCES credit_card(id),
-- Referencia a identificadores unicos
-- de la tabla "company". Cada fila de la tabla
-- "transaction" esta ligada a 0 o 1 filas de "company",
-- ya que la variable es nutable. [1]
company_id VARCHAR(20),
-- Referencia a identificadores unicos
-- de la tabla "user" que aun no hemos creado.
-- Cada fila de la tabla "transaction" esta ligada
-- a 0 o 1 filas de "user", ya que
-- la variable es nutable.
user_id INT REFERENCES user(id),
-- La latitud geografica donde ocurrio la transaccion.
lat FLOAT,
-- La longitud geografica donde ocurrio la transaccion.
longitude FLOAT,
-- Indica el momento en el tiempo cuando
-- ocurrio la transaccion.
timestamp TIMESTAMP,
-- Indica la cantidad de dinero movido
-- en esta transaccion.

```

```

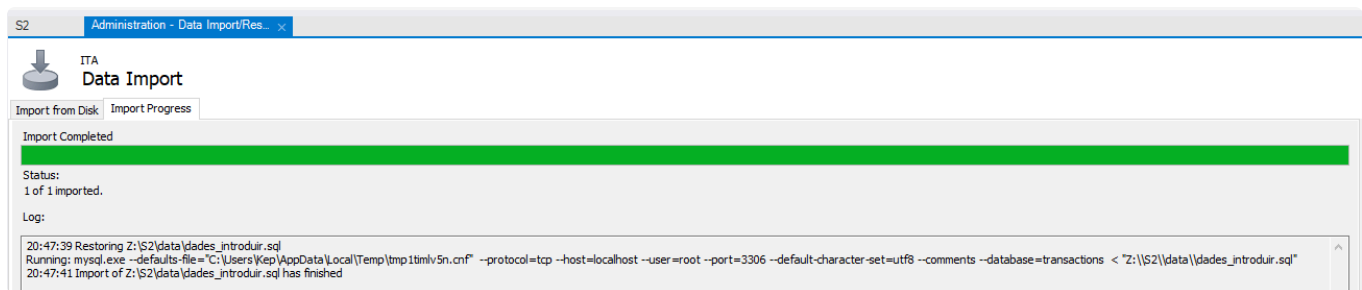
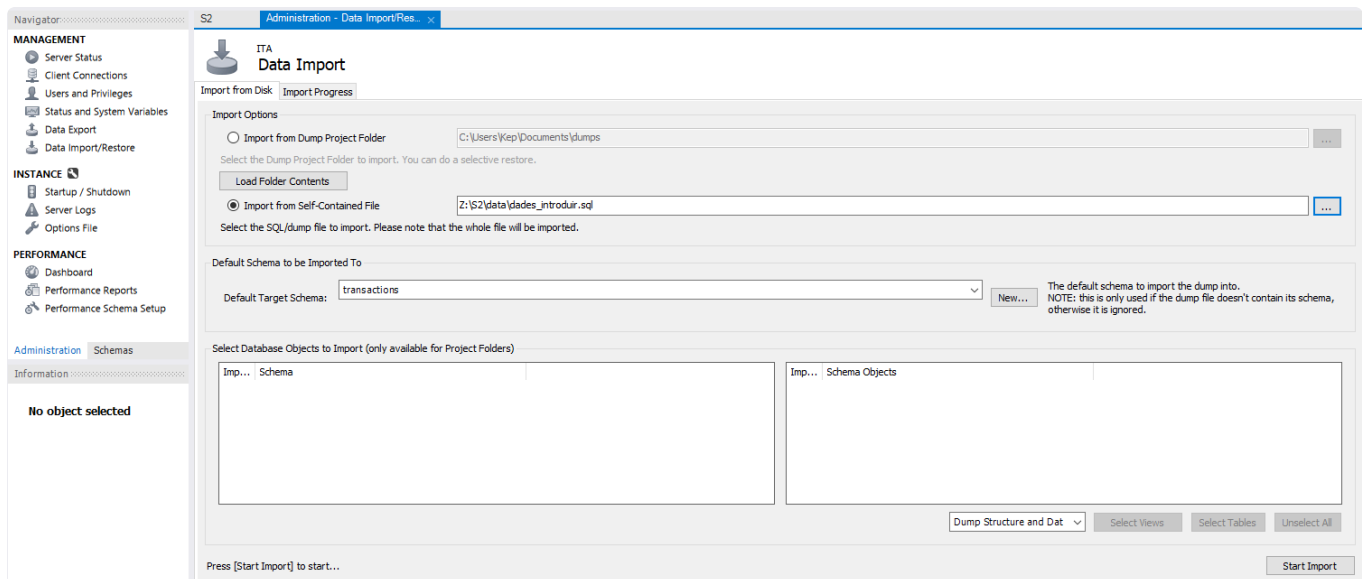
amount DECIMAL(10, 2),
-- Indica si la transaccion a sido rechazada.
-- Si ha sido rechazada, podemos dar por entendido que
-- no hubo venta / la venta fallo.
declined BOOLEAN,
-- Constraint que liga la variable "company_id"
-- al identificador unico "id" de la tabla "company"
FOREIGN KEY (company_id) REFERENCES company(id)
);

```

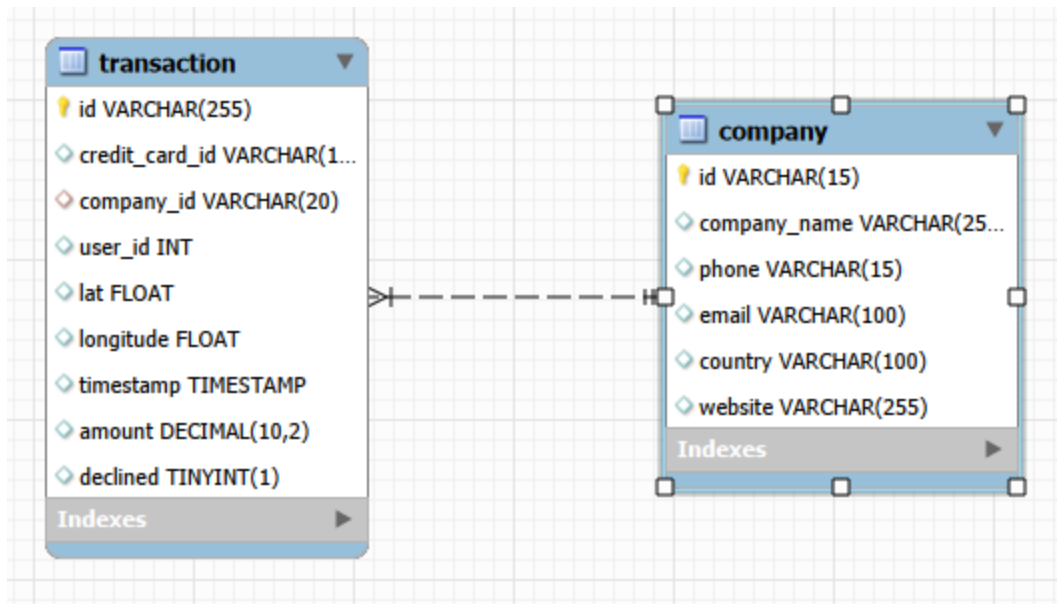
#	Time	Action	Message	Duration / Fetch
1	20:42:47	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.015 sec
2	20:42:51	USE transactions	0 row(s) affected	0.000 sec
3	20:42:57	CREATE TABLE IF NOT EXISTS company (-- Identificador unico de cada fila en la tabla id VARCHAR(15) PRIMARY KEY, -- Nombre de la em...	0 row(s) affected	0.047 sec
4	20:43:01	CREATE TABLE IF NOT EXISTS transaction (-- Identificador unico de cada fila en la tabla id VARCHAR(255) PRIMARY KEY, -- Referencia a id...	0 row(s) affected	0.047 sec

Importar los datos de `dades_introducir.sql` :

1. Panel izquierdo Navigator »
2. Data Import/Restore »
3. Import from Self-Contained File »
4. Introduce la dirección donde se encuentra el fichero.
5. Default Schema to be Imported To »
6. Selecciona el SCHEMA `transactions`
7. Clic Start Import
8. Import Progress Tab »
9. Comprueba en el log que la operación se haya completado sin errores.
10. En el menú de arriba, Tab Database »
11. Clic Reverse Engineer
12. Configura la conexión.
13. Next » Next »
14. Selecciona el SCHEMA `transactions`
15. Next » Execute » Next » Finish
16. Tab EER Diagram deberá mostrar el diagrama de los datos que acabamos de cargar.



Esquema



Descripción adicional

Simple esquema estrella con dos tablas:

- transaction:

- Es la tabla de hechos central del esquema estrella, ya que enlaza tablas de dimensión con sus FK.
- Cada fila representa una transacción monetaria.
- `company` :
 - Es una tabla de dimensión, ya que tiene datos varios de la empresa que representa, y no tiene FK.

Inicialmente, no considere necesario mencionar... Este esquema obviamente refleja dos clases de actores: transacciones, y empresas. Según contexto se puede deducir que las transacciones representan intercambios monetarios entre una empresa y un usuario. En caso de que estos no sean denegados, son ventas. Lógicamente, un usuario solo puede transferir dinero a una empresa. Si fuera necesario transferir a más de una, sería necesario realizar más de una transacción. Por otro lado, la empresa lógicamente puede tener múltiples transferencias de diferentes usuarios. Este modelo refleja perfectamente esta realidad, donde una fila de transacción puede estar enlazada a solo una empresa, y una fila de la tabla de empresas puede estar enlazada a múltiples filas de la tabla de transacciones. Si ampliamos el modelo con los datos que le faltan, la misma relación lógica funciona entre transacciones y usuarios, si sustituimos el comodín empresas que hemos empleado en esta explicación por usuarios. Igual pasaría con tarjetas de crédito: una tarjeta se puede usar en múltiples transacciones, pero una transacción no puede ocurrir desde más o menos de una tarjeta. Se ha de mencionar adicionalmente que este modelo SQL no es lo suficientemente estricto, ya que hay variables anulables y seguramente otras brechas en coherencia e integridad de datos; la más fácil e inmediata de solucionar sería la adición de restricciones `NOT NULL` en las `FOREIGN KEY`.

Relaciones

En los comentarios de código he codificado las relaciones con [N]

```
[1] transaction 0..N <-> 1 company
```

Ex. 2

Una venta constituye una transacción no denegada.

Llistat dels països que estan generant vendes.

```
-- [OP005]
SELECT DISTINCT c.country
FROM transaction AS t
JOIN company AS c
ON t.company_id = c.id
WHERE t.declined = FALSE;
```

country

Germany

Australia

United States

New Zealand

Norway

United Kingdom

Italy

Belgium

Sweden

Result 1 x

Result Grid

Form Editor

Read Only

Context Help

Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	20:42:51	USE transactions	0 row(s) affected	0.000 sec
3	20:42:57	CREATE TABLE IF NOT EXISTS company (-- Identificador unico de cada fila en la tabla id VARCHAR(15) PRIMARY KEY, -- Nombre de la ...	0 row(s) affected	0.047 sec
4	20:43:01	CREATE TABLE IF NOT EXISTS transaction (-- Identificador unico de cada fila en la tabla id VARCHAR(255) PRIMARY KEY, -- Referencia ...	0 row(s) affected	0.047 sec
5	20:49:09	SELECT DISTINCT c.country FROM transaction AS t JOIN company AS c ON t.company_id = c.id	15 row(s) returned	0.000 sec / 0.000 sec

Activate Windows

Go to Settings to activate Windows

Des de quants països es generen les vendes?

```
-- [OP006]
SELECT COUNT(DISTINCT c.country)
FROM transaction AS t
JOIN company AS c
ON t.company_id = c.id
WHERE t.declined = FALSE;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

COUNT(DISTINCT c.country)

15

Result Grid

Read Only

Context Help

Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
3	20:42:57	CREATE TABLE IF NOT EXISTS company (-- Identificador unico de cada fila en la tabla id VARCHAR(15) PRIMARY KEY, -- Nombre de la ...	0 row(s) affected	0.047 sec
4	20:43:01	CREATE TABLE IF NOT EXISTS transaction (-- Identificador unico de cada fila en la tabla id VARCHAR(255) PRIMARY KEY, -- Referencia ...	0 row(s) affected	0.047 sec
5	20:49:09	SELECT DISTINCT c.country FROM transaction AS t JOIN company AS c ON t.company_id = c.id	15 row(s) returned	0.000 sec / 0.000 sec
6	20:50:23	SELECT COUNT(DISTINCT c.country) FROM transaction AS t JOIN company AS c ON t.company_id = c.id	1 row(s) returned	0.016 sec / 0.000 sec

Identifica la companyia amb la mitjana més gran de vendes.

```
-- [OP007]
SELECT
  c.id,
  c.company_name,
  ROUND(AVG(t.amount), 2) AS media
FROM transaction AS t
JOIN company AS c
ON t.company_id = c.id
WHERE t.declined = FALSE
GROUP BY c.id
ORDER BY media DESC
LIMIT 1;
```

Result Grid			Filter Rows	Export	Wrap Cell Content	15	
id	company_name	media					Result Grid
b-2398	Eget Ipsum Ltd	481.86					

Result 3		Read Only	Context Help	Snippets
Output				
#	Time	Action	Message	Duration / Fetch
4	20:43:01	CREATE TABLE IF NOT EXISTS transaction (-- Identificador unico de cada fila en la tabla id VARCHAR(255) PRIMARY KEY, -- Referencia ...	0 row(s) affected	0.047 sec
5	20:49:09	SELECT DISTINCT c.country FROM transaction AS t JOIN company AS c ON t.company_id = c.id	15 row(s) returned	0.000 sec / 0.000 sec
6	20:50:23	SELECT COUNT(DISTINCT c.country) FROM transaction AS t JOIN company AS c ON t.company_id = c.id	1 row(s) returned	0.016 sec / 0.000 sec
7	20:50:58	SELECT c.id, c.company_name, ROUND(AVG(t.amount), 2) AS media FROM transaction AS t JOIN company AS c ON t.company_id = c.id ...	1 row(s) returned	0.000 sec / 0.000 sec

Según mi consulta de documentación SQL, se considera buena práctica hacer un GROUP BY de todas las variables no resumidas (AVG en este caso), por motivos de compatibilidad.

En el P2P se me ha comentado que mi mentora Alana prefiere los GROUP BY ajustados, por eso todas las operaciones en este sprint tendrán GROUP BY minimalistas.

Gracias por la aclaración, Alana Olivieri. Un GROUP BY ajustado no solo es preferencia personal: a pesar de lo que se diga en buenas prácticas, y lo que practicamos en fundamentos con Oriol Tinoco, mantener el GROUP BY ajustado elimina redundancia y, por tanto, puede acelerar y hacer las consultas más eficientes.

En entornos de producción, siempre es preferente minimizar el ancho y redundancia de las operaciones, como bien menciona Alana en su corrección.

También hace falta mencionar que dependiendo del sistema de base de datos, un GROUP BY amplio puede ser necesario.

Ex. 3

Utilitzant només subconsultes (sense utilitzar JOIN):

Mostra totes les transaccions realitzades per empreses d'Alemanya.

```
-- [OP008]
SELECT *
FROM transaction AS t
WHERE declined = FALSE
AND EXISTS (
    SELECT 1
    FROM company as c
    WHERE t.company_id = c.id
    AND c.country = "Germany"
);
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
10881D-5B23-A76C-55EF-C568E49A05D0	CcJ-2938	b-2222	275	83.7839	-178.86	2021-07-07 17:43:16	293.57	0
AB069F53-965E-A2A8-CE06-CABC4FD92501	CcJ-2959	b-2234	275	1.64819	-158.007	2021-04-15 13:37:18	60.99	0
0466A42E-47CF-8024-FD01-C0689713128	CcJ-4219	b-2302	170	-43.9695	-117.525	2021-07-26 07:29:18	49.53	0
0A476ED9-9C13-1962-F87B-D35639246539	CcJ-4359	b-2302	221	-56.4901	114.801	2022-02-26 20:33:54	430.49	0
1220C333-E19F-0629-DCD6-9C54CFEE89A	CcJ-4366	b-2302	221	29.6372	-166.173	2021-06-09 06:04:14	172.01	0
1332678A-2E7D-907C-4C0C-6450A2B3D354	CcJ-4920	b-2302	210	20.6724	14.9732	2021-12-29 20:38:23	17.97	0
14CAE9B5-8F81-3E4A-4C85-0EA4167534F4	CcJ-4849	b-2302	189	-53.6202	93.0533	2021-12-31 00:29:42	388.04	0
158A3ACB-541C-08CC-658D-6373CC678F1C	CcJ-4849	b-2302	183	40.5424	-170.347	2022-03-08 05:02:19	240.29	0
162C7E78-2B6E-7971-A1E4-0212-E732451	CcJ-4527	b-2302	210	-69.1381	58.0017	2021-04-11 05:59:18	231.26	0
1717F06B-ADAD-7082-A748-91128E892CCC	CcJ-4219	b-2302	172	69.4892	-138.411	2021-12-29 16:18:54	249.91	0
1753A288-9FC1-52E6-5C39-A1FFB978D03A	CcJ-4345	b-2302	222	57.9422	-114.729	2021-08-17 05:32:08	497.84	0
186F53DE-DE27-B1FE-882F-15861CEB7726	CcJ-4310	b-2302	225	-72.7448	36.6211	2021-12-20 13:13:45	238.16	0
18C4E2D0-1E4C-F35E-2198-C660B081DC25	CcJ-4849	b-2302	177	68.0133	91.4839	2021-09-24 18:55:25	237.04	0
18CBA7C-ABC1-813D-FAF3-80A897A29368	CcJ-4219	b-2302	173	51.3881	-156.371	2021-06-21 03:21:34	58.16	0
19E1EC3E-2119-1F0A-6A4E-593D04A4E53F	CcJ-4219	b-2302	154	-56.0839	116.987	2021-07-06 21:40:15	29.63	0
1B117D49-936C-8A6C-E94B-30C3293AA239	CcJ-4219	b-2302	163	-75.099	109.034	2021-05-09 23:56:04	371.35	0
1B521826-5860-5A86-5364-6EB6A5CC21B7	CcJ-4226	b-2302	231	-53.4613	49.1484	2021-08-14 02:39:50	476.33	0
1C383CC4-B919-7616-6A57-EEBDD7B76888	CcJ-4415	b-2302	217	81.1737	62.8835	2021-09-27 16:07:34	462.35	0
2075812D-69AB-7022-578B-33ED49672766	CcJ-4247	b-2302	229	-9.03682	88.9005	2022-02-13 04:07:29	87.44	0

Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

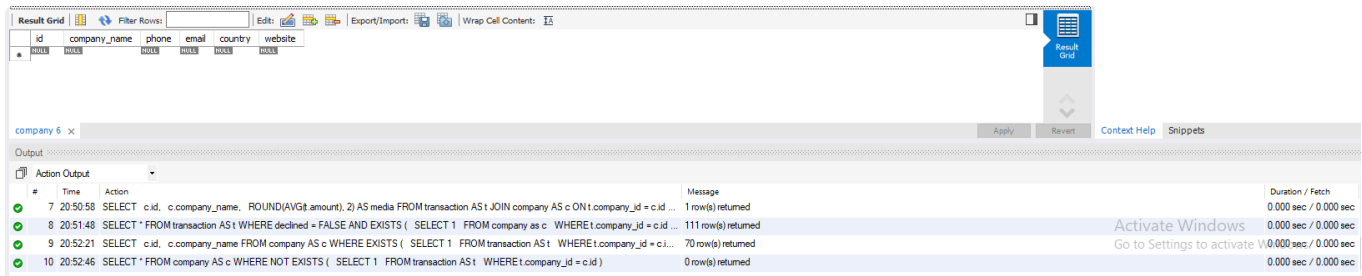
```
-- [0P009]
SELECT
  c.id,
  c.company_name
FROM company AS c
WHERE EXISTS (
  SELECT 1
  FROM transaction AS t
  WHERE t.company_id = c.id
  AND t.amount > (
    SELECT AVG(t.amount)
    FROM transaction AS t
  )
);
```

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Nique Industries
b-2230	Fusce Corp.
b-2238	Ante Iaculis Nec Foundation
b-2242	Donec Ltd
b-2246	Sed Nunc Ltd
b-2254	Nascetur Ridiculus Mus Inc.
b-2258	Vestibulum Lorem PC
b-2262	Gravida Sagittis LLP
b-2266	Mus Aenean Eget Foundation
b-2270	Dis Parturient Institute
b-2274	Sed LLC
b-2278	Arcu LLP
b-2286	Fringilla LLC
b-2298	Elit Etiam Laoreet Associates
b-2302	Nunc Interdum Incorporated
b-2306	Augue Foundation
b-2310	Non Magna LLC
b-2314	A Institute

Elimina del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Lista de empresas sin transacciones. Todas las empresas tienen transacciones, la lista está vacía.

```
-- [OP010]
SELECT *
FROM company AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM transaction AS t
    WHERE t.company_id = c.id
);
```



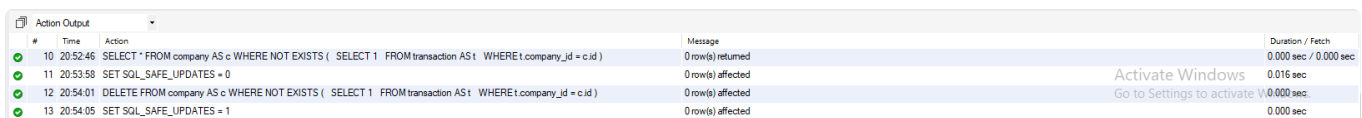
id	company_name	phone	email	country	website
----	--------------	-------	-------	---------	---------

```
-- [OP012] Temporarily disable Safe Mode
SET SQL_SAFE_UPDATES = 0;
```

Esta operación elimina 0 empresas, ya que todas las empresas tienen transacciones.

```
-- [OP011]
DELETE FROM company AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM transaction AS t
    WHERE t.company_id = c.id
);
```

```
-- [OP013] Return to Safe Mode
SET SQL_SAFE_UPDATES = 1;
```



#	Time	Action	Message	Duration / Fetch
10	20:52:46	SELECT * FROM company AS c WHERE NOT EXISTS (SELECT 1 FROM transaction AS t WHERE t.company_id = c.id)	0 row(s) returned	0.000 sec / 0.000 sec
11	20:53:58	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.016 sec
12	20:54:01	DELETE FROM company AS c WHERE NOT EXISTS (SELECT 1 FROM transaction AS t WHERE t.company_id = c.id)	0 row(s) affected	0.000 sec
13	20:54:05	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec

Ex. 4

La teva tasca és dissenyar i crear una taula anomenada `credit_card` que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat `dades_introduir_credit`. Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

```
-- [OP014]
DROP TABLE IF EXISTS credit_card;
```

```
-- [OP015]
CREATE TABLE IF NOT EXISTS credit_card (
  -- Identificador unico de cada fila de la tabla.
  id VARCHAR(15) PRIMARY KEY,
  -- El iban de la tarjeta.
  iban VARCHAR(50),
  -- El pan de la tarjeta.
  pan VARCHAR(30),
  -- El pin de la tarjeta.
  pin VARCHAR(10),
  -- El cvv de la tarjeta.
  cvv VARCHAR(10),
  -- La data de caducidad de la tarjeta.
  expiring_date VARCHAR(10)
);
```

Importar los datos de `N1-Ex.4__datos_introducir_credit.sql`:

1. Panel izquierdo Navigator »
 2. Data Import/Restore »
 3. Import from Self-Contained File »
 4. Introduce la dirección donde se encuentra el fichero.
 5. Default Schema to be Imported To »
 6. Selecciona el SCHEMA `transactions`
 7. Clic Start Import
 8. Import Progress Tab »
 9. Comprueba en el log que la operación se haya completado sin errores.
-

S2 Administration - Data Import/Res...

ITA Data Import

Import from Disk Import Progress

Import Options

☐ Import from Dump Project Folder C:\Users\Kep\Documents\dumps ...

Select the Dump Project Folder to import. You can do a selective restore.

☒ Import from Self-Contained File Z:\S2\data\W1-Ex.4__datos_introducir_credit.sql ...

Select the SQL/dump file to import. Please note that the whole file will be imported.

Default Schema to be Imported To

Default Target Schema: transactions

The default schema to import the dump into.
NOTE: this is only used if the dump file doesn't contain its schema, otherwise it is ignored.

Select Database Objects to Import (only available for Project Folders)

Imp...	Schema
--------	--------

Imp...	Schema Objects
--------	----------------

Import Completed

S2 Administration - Data Import/Res...

ITA Data Import

Import from Disk Import Progress

Import Completed

Status:
1 of 1 imported.

Log:

```

20:57:10 Restoring Z:\S2\data\ldades_introduir.sql
Running: mysql.exe -defaults-file="C:\Users\Kep\AppData\Local\Temp\lmp3h8twzxx.cnf" --protocol=tcp --host=localhost --user=root --port=3306 --default-character-set=utf8 --comments --database=transactions < "Z:\S2\data\ldades_introduir.sql"
20:57:13 Import of Z:\S2\data\ldades_introduir.sql has finished

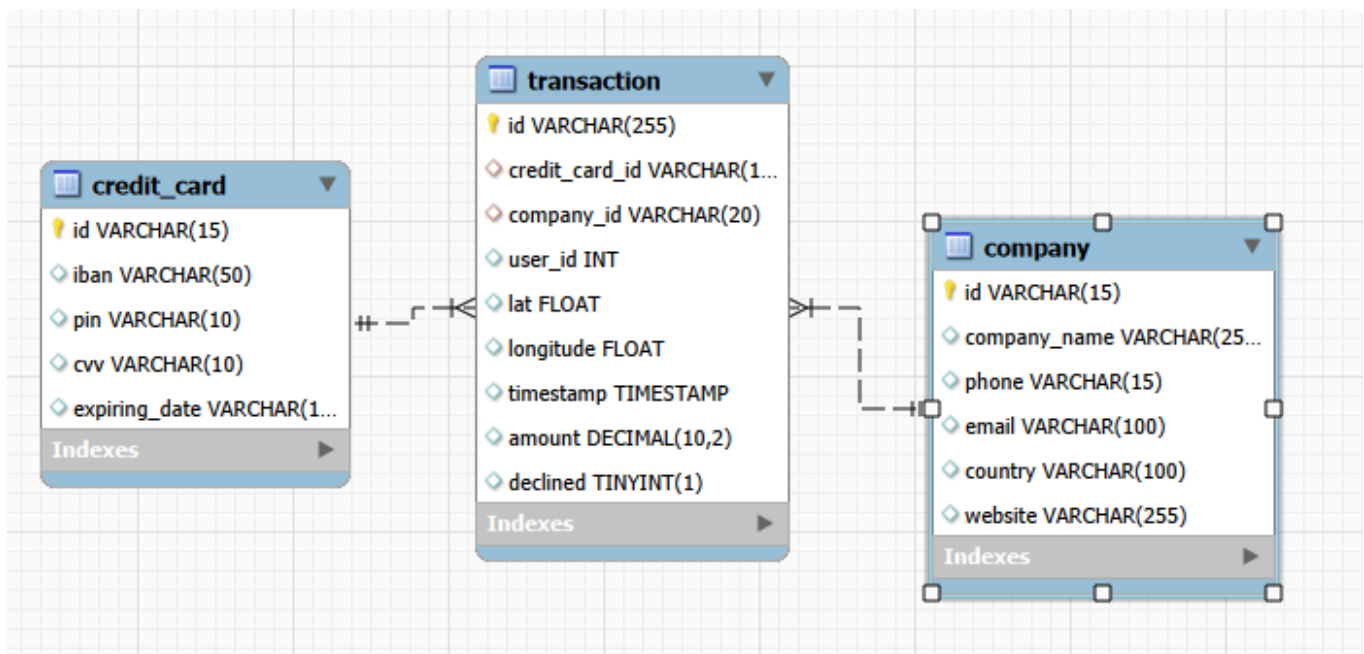
20:58:58 Restoring Z:\S2\data\W1-Ex.4__datos_introducir_credit.sql
Running: mysql.exe -defaults-file="C:\Users\Kep\AppData\Local\Temp\lmp70ko_azz.cnf" --protocol=tcp --host=localhost --user=root --port=3306 --default-character-set=utf8 --comments --database=transactions < "Z:\S2\data\W1-Ex.4__datos_introducir_credit.sql"
20:59:11 Import of Z:\S2\data\W1-Ex.4__datos_introducir_credit.sql has finished
  
```

Añadimos el FOREIGN KEY CONSTRAINT a la tabla transaction, ya que hemos creado la tabla.

```
-- [OP016]
ALTER TABLE transaction
ADD CONSTRAINT fk_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

1. En el menú de arriba, Tab Database »
2. Clic Reverse Engineer
3. Configura la conexión.
4. Next » Next »
5. Selecciona el SCHEMA transactions
6. Next » Execute » Next » Finish
7. Tab EER Diagram deberá mostrar el diagrama de los datos que acabamos de cargar.

Esquema



Mostramos la tabla `credit_card`:

```
-- [OP017]
SELECT *
FROM credit_card;
```

id	iban	pan	pin	cvv	expiring_date
CS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
CS-4860	XX559590368835304645299	2481155515498459	6876	661	07/27/27
CS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26
CS-4862	XX474721462463645409798	6715617009807825	4042	174	11/27/26
CS-4863	XX147829664245046207111	3140879819451394	5969	449	12/27/29
CS-4864	XX8380298893385731196159	5793672133649114	8481	139	02/28/26
CS-4865	XX7085078596101025280599	5101552687251312	7847	903	11/25/28
CS-4866	XX4792859188206596406839	8080768801072613	9271	961	02/28/25
CS-4867	XX6038298816319374853717	7761849537661098	4820	862	11/30/25
CS-4868	XX3929101617300533068044	4160419663151801	6860	549	08/25/25
CS-4869	XX1566712096111531886465	5892930716233310	4775	448	09/29/25
CS-4870	XX6443663804167732133949	2227240879996178	5872	715	01/25/25
CS-4871	XX353020958376786066563	694414245556838	7992	643	07/26/26
CS-4872	XX40916211403158197546334	5035418997919499	3642	191	12/29/26
CS-4873	XX15214007031123575311011	27303700462640752	1038	173	04/16/26

Descripción adicional

Simple esquema estrella con dos tablas:

- `transaction` :
 - Es la tabla de hechos central del esquema estrella, ya que enlaza tablas de dimensión con sus FK.
 - Cada fila representa una transacción monetaria.
- `company` :
 - Es una tabla de dimensión, ya que tiene datos varios de la empresa que representa, y no tiene FK.
- `credit_card` :
 - Es una tabla de dimension, ya que tiene datos variados de la tarjeta que representa, y no tiene FK.

Relaciones

```
[1] transaction 0..N <-> 1 company
[2] transaction 0..N <-> 1 credit_card
```

Ex. 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938 . La informació que ha de mostrar-se per a aquest registre és: TR3234563122135768176999999 .

Recorda mostrar que el canvi es va realitzar.

Observamos la fila antes del cambio tiene el iban TR301950312213576817638661 .

```
-- [OP018]
SELECT *
```

```
FROM credit_card AS c
WHERE c.id = "CcU-2938";
```

The screenshot shows a database management tool interface. At the top, there's a 'Result Grid' with a table containing 8 rows. The columns are: id, iban, pan, pin, cvv, and expiring_date. The first row is highlighted. Below the grid, there's a tab labeled 'credit_card 8'. Underneath, there's an 'Output' section with a dropdown menu set to 'Action Output'. The output shows a log of database actions with columns: #, Time, Action, Message, and Duration / Fetch. The log shows several actions, including a CREATE TABLE statement, an ALTER TABLE statement adding a foreign key constraint, and a SELECT statement that returned 1 row.

#	Time	Action	Message	Duration / Fetch
15	20:57:59	CREATE TABLE IF NOT EXISTS credit_card (-- Identificador unico de cada fila de la tabla. id VARCHAR(15) PRIMARY KEY, -- El iban de l...	0 row(s) affected	0.031 sec
16	20:59:58	ALTER TABLE transaction ADD CONSTRAINT fk_card FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	587 row(s) affected Records: 587 Duplicates: 0 Warnings: 0	0.109 sec
17	21:00:05	SELECT * FROM credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
18	21:01:27	SELECT * FROM credit_card AS c WHERE c.id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

Efectuamos el cambio pedido.

```
-- [0P019]
UPDATE credit_card AS c
SET iban = "TR323456312213576817699999"
WHERE c.id = "CcU-2938";
```

Comprobamos si el cambio se ha realizado:

```
-- [0P020]
SELECT *
FROM credit_card AS c
WHERE c.id = "CcU-2938";
```

The screenshot shows the same database management tool interface. The 'Result Grid' now shows a single row with the updated iban. The 'Output' section shows a log of database actions. The log shows a SELECT statement that returned 5000 rows, followed by a SELECT statement that returned 1 row, and then an UPDATE statement that affected 1 row.

#	Time	Action	Message	Duration / Fetch
17	21:00:05	SELECT * FROM credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
18	21:01:27	SELECT * FROM credit_card AS c WHERE c.id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
19	21:02:04	UPDATE credit_card AS c SET iban = "TR323456312213576817699999" WHERE c.id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
20	21:02:06	SELECT * FROM credit_card AS c WHERE c.id = "CcU-2938"	1 row(s) returned	0.016 sec / 0.000 sec

Ex. 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:

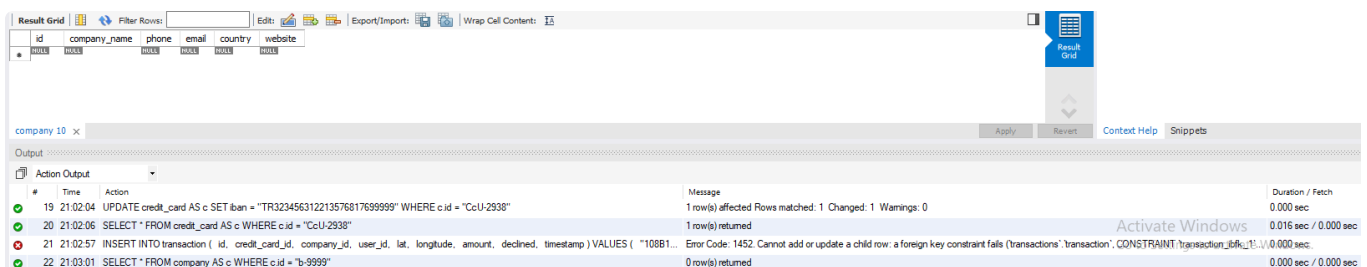
```
id : 108B1D1D-5B23-A76C-55EF-C568E49A99DD
credit_card_id : CcU-9999
company_id : b-9999
user_id : 9999
lat : 829.999
longitude : -117.999
amount : 111.11
declined : 0
```

Esta operacion va a fallar:

```
-- [OP021]
INSERT INTO transaction (
  id,
  credit_card_id,
  company_id,
  user_id,
  lat,
  longitude,
  amount,
  declined,
  timestamp
) VALUES (
  "108B1D1D-5B23-A76C-55EF-C568E49A99DD",
  "CcU-9999",
  "b-9999",
  9999,
  829.999,
  -117.999,
  111.11,
  0,
  NOW()
);
```

Comprobamos que no existe una compañía con ID b-9999

```
-- [OP022]
SELECT *
FROM company AS c
WHERE c.id = "b-9999";
```



#	Time	Action	Message	Duration / Fetch
19	21:02:04	UPDATE credit_card AS c SET iban = "TR323456312213576817699999" WHERE c.id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
20	21:02:06	SELECT * FROM credit_card AS c WHERE c.id = "CcU-2938"	1 row(s) returned	0.016 sec / 0.000 sec
21	21:02:57	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined, timestamp) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CcU-9999", "b-9999", 9999, 829.999, -117.999, 111.11, 0, NOW())	Error Code: 1452 Cannot add or update a child row: a foreign key constraint fails (transactions.transaction, CONSTRAINT transaction_ibfk_1:...	0.000 sec
22	21:03:01	SELECT * FROM company AS c WHERE c.id = "b-9999"	0 row(s) returned	0.000 sec / 0.000 sec

No puedo proceder sin consultar: necesito datos adicionales sobre la tarjeta de crédito y compañía involucrados en esta transacción. No puedo inventármelos. No puedo quitar las FOREIGN KEY CONSTRAINT s, ya que tengo que mantener la integridad de los datos.

Ex. 7

Des de recursos humans et sol·liciten eliminar la columna `pan` de la taula `credit_card` .
Recorda mostrar el canvi realitzat.

Observamos la tabla `credit_card` :

```
-- [OP023]
SELECT *
FROM credit_card
LIMIT 10;
```

The screenshot shows a database management tool interface. The top part displays a 'Result Grid' with columns: id, iban, pan, pin, cvv, and expiring_date. Below this, the 'Action Output' tab is active, showing a log of database actions. The log includes a SELECT query (20), an INSERT query (21) that failed with an error, and a SELECT query (22) that returned 0 rows. The final action (23) is a SELECT query that returned 10 rows.

#	Time	Action	Message	Duration / Fetch
20	21.02.06	SELECT * FROM credit_card AS c WHERE c.id = "CcU-2938"	1 row(s) returned	0.016 sec / 0.000 sec
21	21.02.57	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined, timestamp) VALUES ("108B1..."	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (transactions.transaction, CONSTRAINT transaction_fc...	0.000 sec
22	21.03.01	SELECT * FROM company AS c WHERE c.id = "b-9999"	0 row(s) returned	0.000 sec / 0.000 sec
23	21.03.46	SELECT * FROM credit_card LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Eliminamos la columna como se nos ha pedido:

```
-- [OP024]
ALTER TABLE credit_card
DROP COLUMN pan;
```

Comprobamos que se ha realizado el cambio:

```
-- [OP025]
SELECT *
FROM credit_card
LIMIT 10;
```

The screenshot shows the same database management tool interface. The 'Result Grid' now only displays columns: id, iban, pin, cvv, and expiring_date, as the 'pan' column has been successfully dropped. The 'Action Output' tab shows the log of actions, including the successful execution of the DROP COLUMN query (24).

#	Time	Action	Message	Duration / Fetch
22	21.03.01	SELECT * FROM company AS c WHERE c.id = "b-9999"	0 row(s) returned	0.000 sec / 0.000 sec
23	21.03.46	SELECT * FROM credit_card LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec
24	21.04.19	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
25	21.04.21	SELECT * FROM credit_card LIMIT 10	10 row(s) returned	0.000 sec / 0.000 sec

Ex. 8

Descarrega els arxius CSV:

```
american_users.csv  
european_users.csv  
companies.csv  
credit_cards.csv  
transactions.csv
```

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

La taula de `products.csv` l'utilitzarem més endavant.

1. Crear la carpeta: `C:\Program Files\MySQL\MySQL Server 8.0\Uploads`
2. Mover los ficheros a la carpeta.
3. Configurar la conexi3n del `workbench`.
4. `advanced` »
5. Escribe en el text field `others` : `OPT_LOCAL_INFILE=1`

```
-- [OP026]  
CREATE SCHEMA IF NOT EXISTS S2L1E8;
```

```
-- [OP027]  
USE S2L1E8;
```

```
-- [OP028]  
DROP TABLE IF EXISTS transactions;
```

```
-- [OP029]  
DROP TABLE IF EXISTS products;
```

```
-- [OP030]  
DROP TABLE IF EXISTS cards;
```

```
-- [OP031]
```

```
DROP TABLE IF EXISTS users;
```

```
-- [OP032]  
DROP TABLE IF EXISTS companies;
```

Creamos las tablas:

```
-- [OP033]  
CREATE TABLE IF NOT EXISTS users (  
  -- Identificador unico de cada fila  
  id          INT PRIMARY KEY,  
  -- Nombre  
  name        VARCHAR(20),  
  -- Apellido  
  surname     VARCHAR(20),  
  -- Numero de telefono  
  phone       VARCHAR(20),  
  -- Email  
  email       VARCHAR(80),  
  -- Data de nacimiento  
  birth_date  DATE,  
  -- Pais  
  country     VARCHAR(20),  
  -- Ciudad  
  city        VARCHAR(20),  
  --Codigo postal  
  postal_code VARCHAR(10),  
  -- Direccion  
  address     VARCHAR(100),  
  -- Data de registro  
  signup_date DATE,  
  -- Categoria / clasificacion de comprador  
  segment     VARCHAR(40),  
  -- Categoria de sueldo  
  income_band VARCHAR(20)  
);
```

```
-- [OP034]  
CREATE TABLE IF NOT EXISTS cards (  
  -- Identificador unico de cada fila  
  id          VARCHAR(15) PRIMARY KEY,  
  -- Referencia a identificadores unicos de la  
  -- tabla "users". Una instancia de "cards"  
  -- puede ser enlazada a 0 o 1 instancias
```

```

-- de "users", ya que "user_id" es nutable. [1]
user_id      INT,
-- El iban de la tarjeta que representa la fila.
iban         VARCHAR(50),
-- El pan de la tarjeta.
pan          VARCHAR(30),
-- El pin de la tarjeta.
pin          VARCHAR(8),
-- El cvv de la tarjeta.
cvv          VARCHAR(8),
-- La banda magnetica uno de la tarjeta.
track1       VARCHAR(80),
-- La banda magnetica dos de la tarjeta.
track2       VARCHAR(40),
-- La data de expiracion.
expiring_date VARCHAR(10),
-- El tipo de tarjeta.
card_type    VARCHAR(20),
-- Si ha sido renovada.
renewal_flag BOOL,
-- Esto programa el constraint FK [1]
FOREIGN KEY (user_id) REFERENCES users(id)
);

```

```

-- [OP035]
CREATE TABLE IF NOT EXISTS companies (
-- Identificador unico de cada fila.
id          VARCHAR(15) PRIMARY KEY,
-- El nombre de la empresa.
name        VARCHAR(50),
-- Numero de telefono.
phone       VARCHAR(20),
-- Email
email       VARCHAR(80),
-- Pais
country     VARCHAR(20),
-- Pagina web
website     VARCHAR(60),
-- Categoria de producto
category    VARCHAR(20),
-- Categoria de market (alto, medio, barato)
-- al cual dirige sus productos.
price       VARCHAR(20)
);

```

```

-- [OP036]
CREATE TABLE IF NOT EXISTS products (
  -- Identificador unico de cada fila.
  id          INT PRIMARY KEY,
  -- Nombre del producto.
  name        VARCHAR(40),
  -- Precio.
  price       DECIMAL(8,2),
  -- Color RGB en notacion hexadecimal
  hex_color   CHAR(6),
  -- El peso del producto
  weight      DECIMAL(8,2),
  -- La id del almacen
  warehouse_id INT,
  -- La categoria del producto
  category    VARCHAR(20),
  -- La marka del producto
  brand       VARCHAR(20),
  -- Cuanto costo adquirir/producir el producto a la empresa.
  cost        DECIMAL(8,2),
  -- La data cuando el producto se puso en venta.
  launch_date DATE
);

```

```

-- [OP037]
CREATE TABLE IF NOT EXISTS transactions (
  -- Identificador unico de cada fila.
  id          VARCHAR(40) PRIMARY KEY,
  -- [2] Enlaza con el identificador unico "id"
  -- de la tabla "cards".
  card_id     VARCHAR(15),
  -- [3] Enlaza con el identificador unico "id"
  -- de la tabla "company".
  company_id  VARCHAR(15),
  -- Indica el momento en el tiempo
  -- cuando ocurrio la transaccion.
  instant     TIMESTAMP,
  -- Indica la cantidad de dinero transferido.
  amount      DECIMAL(8,2),
  -- Indica si se ha denegado la transaccion.
  declined    BOOL,
  -- Una lista de ids que en teoria enlazan
  -- con identificadores unicos de la tabla
  -- "products". Se tienen que procesar para
  -- ser usables en nuestra base de datos.

```

```

product_ids    VARCHAR(255),
-- [4] Enlaza con el identificador unico "id"
-- de la tabla "users".
user_id        INT,
-- Indica la latitud geografica
-- donde ocurrio la transaccion.
latitude       FLOAT,
-- Indica la longitud geografica
-- donde ocurrio la transaccion.
longitude      FLOAT,
-- Indica descuentos monetarios de la transaccion
discount       DECIMAL(8,2),
-- Indica la porcion cargada de impuesto.
tax            DECIMAL(8,2),
-- Indica la porcion cargada de transporte.
shipping       DECIMAL(8,2),
-- Indica el canal donde se ejecuta la transaccion.
-- ("online", "store", "marketplace")
channel        VARCHAR(20),
-- Indica el identificador de "campaign"
-- ("SUMMER_TRAVEL", "WINTER_SALE")
campaign_id    VARCHAR(30),
-- Indica el dispositivo electronico desde el cual
-- se realizo la transaccion.
device_type    VARCHAR(20),
-- Indica si la transaccion es internacional.
international  BOOL,
-- Si la transaccion fue denegada, aqui se indica la razon.
decline_reason VARCHAR(255),
-- Supongo que es la distancia del transporte a "domicilio".
distance_km    DECIMAL(8,2),
-- Esto programa el constraint FK [3]
FOREIGN KEY (company_id) REFERENCES companies(id),
-- Esto programa el constraint FK [4]
FOREIGN KEY (user_id) REFERENCES users(id),
-- Esto programa el constraint FK [2]
FOREIGN KEY (card_id) REFERENCES cards(id)
);

```

Creamos índices para acelerar operaciones `JOIN`. Doy por supuesto que `PRIMARY KEY` ya genera su propio índice por defecto.

```

-- [OP038]
CREATE INDEX idx_cards_user_id
ON cards(user_id);

```

```
-- [OP039]
CREATE INDEX idx_transactions_card_id
ON transactions(card_id);
```

```
-- [OP040]
CREATE INDEX idx_transactions_company_id
ON transactions(company_id);
```

```
-- [OP041]
CREATE INDEX idx_transactions_user_id
ON transactions(user_id);
```

Necesitamos habilitar permisos para cargar los datos.

```
-- [OP042]
SET GLOBAL local_infile = ON;
```

El servidor está restringido a leer ficheros desde la carpeta indicada por el siguiente comando. Observamos que la carpeta es:

- C:\ProgramData\MySQL\MySQL Server 8.0\Uploads

```
-- [OP043]
SHOW VARIABLES LIKE "secure_file_priv";
```

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

#	Time	Action	Message	Duration / Fetch
26	21:05:07	CREATE SCHEMA IF NOT EXISTS S2L1E8	1 row(s) affected	0.016 sec
27	21:05:09	USE S2L1E8	0 row(s) affected	0.000 sec
28	21:05:14	DROP TABLE IF EXISTS transactions	0 row(s) affected, 1 warning(s): 1051 Unknown table 'S2L1E8.transactions'	0.000 sec
29	21:05:17	DROP TABLE IF EXISTS products	0 row(s) affected, 1 warning(s): 1051 Unknown table 'S2L1E8.products'	0.016 sec
30	21:05:18	DROP TABLE IF EXISTS cards	0 row(s) affected, 1 warning(s): 1051 Unknown table 'S2L1E8.cards'	0.000 sec
31	21:05:20	DROP TABLE IF EXISTS users	0 row(s) affected, 1 warning(s): 1051 Unknown table 'S2L1E8.users'	0.000 sec
32	21:05:21	DROP TABLE IF EXISTS companies	0 row(s) affected, 1 warning(s): 1051 Unknown table 'S2L1E8.companies'	0.000 sec
33	21:05:26	CREATE TABLE IF NOT EXISTS users (-- Identificador unico de cada fila. id INT PRIMARY KEY, -- Nombre name VARCHAR(20...	0 row(s) affected	0.062 sec
34	21:05:35	CREATE TABLE IF NOT EXISTS cards (-- Identificador unico de cada fila. id VARCHAR(15) PRIMARY KEY, -- Referencia a identific...	0 row(s) affected	0.047 sec
35	21:05:40	CREATE TABLE IF NOT EXISTS companies (-- Identificador unico de cada fila. id VARCHAR(15) PRIMARY KEY, -- El nombre de la em...	0 row(s) affected	0.032 sec
36	21:05:47	CREATE TABLE IF NOT EXISTS products (-- Identificador unico de cada fila. id INT PRIMARY KEY, -- Nombre del producto. name ...	0 row(s) affected	0.046 sec
37	21:05:52	CREATE TABLE IF NOT EXISTS transactions (-- Identificador unico de cada fila. id VARCHAR(40) PRIMARY KEY, -- [2] Enlaza co...	0 row(s) affected	0.063 sec
38	21:06:00	CREATE INDEX idx_cards_user_id ON cards(user_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
39	21:06:08	CREATE INDEX idx_transactions_card_id ON transactions(card_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
40	21:06:12	CREATE INDEX idx_transactions_company_id ON transactions(company_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
41	21:06:14	CREATE INDEX idx_transactions_user_id ON transactions(user_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
42	21:06:17	SET GLOBAL local_infile = ON	1 row(s) affected	0.000 sec
43	21:06:20	SHOW VARIABLES LIKE "secure_file_priv"	1 row(s) returned	0.000 sec / 0.000 sec

```
LOAD DATA
[LOW_PRIORITY | CONCURRENT] [LOCAL]
INFILE 'file_name'
```

```

[REPLACE | IGNORE]
INTO TABLE tbl_name
[PARTITION (partition_name [, partition_name] ...)]
[CHARACTER SET charset_name]
[{FIELDS | COLUMNS}
  [TERMINATED BY 'string']
  [[OPTIONALLY] ENCLOSED BY 'char']
  [ESCAPED BY 'char']
]
[LINES
  [STARTING BY 'string']
  [TERMINATED BY 'string']
]
[IGNORE number {LINES | ROWS}]
[(col_name_or_user_var
  [, col_name_or_user_var] ...)]
[SET col_name={expr | DEFAULT}
  [, col_name={expr | DEFAULT}] ...]

```

Cargamos datos a la tabla usuarios:

```

-- [OP044]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-
Ex.8__american_users.csv"
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@id, @name, @surname, @phone, @email, @birth, @country, @city,
 @postal, @address, @signup, @segment, @income)
SET
  id          = CAST(@id AS UNSIGNED),
  name        = TRIM(@name),
  surname     = TRIM(@surname),
  phone       = TRIM(@phone),
  email       = TRIM(@email),
  birth_date  = STR_TO_DATE(@birth, '%b %d, %Y'),
  country     = TRIM(@country),
  city        = TRIM(@city),
  postal_code = TRIM(@postal),
  address     = TRIM(@address),
  signup_date = STR_TO_DATE(@signup, '%Y-%m-%d'),
  segment     = TRIM(@segment),

```

```
income_band = TRIM(@income)
;
```

```
-- [OP045]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-
Ex.8__european_users.csv"
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@id, @name, @surname, @phone, @email, @birth, @country, @city,
 @postal, @address, @signup, @segment, @income)
SET
  id          = CAST(@id AS UNSIGNED),
  name        = TRIM(@name),
  surname     = TRIM(@surname),
  phone       = TRIM(@phone),
  email       = TRIM(@email),
  birth_date  = STR_TO_DATE(@birth, '%b %d, %Y'),
  country     = TRIM(@country),
  city        = TRIM(@city),
  postal_code = TRIM(@postal),
  address     = TRIM(@address),
  signup_date = STR_TO_DATE(@signup, '%Y-%m-%d'),
  segment     = TRIM(@segment),
  income_band = TRIM(@income)
;
```

Comprobamos que los datos se cargaron:

```
-- [OP046]
SELECT * FROM users;
```

Result Grid												
id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	segment	income_band
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	1985-11-17	United States	New York	10001	348-7818 Sagittis St.	2014-04-23	young_professional	high
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	1992-08-23	United States	Philadelphia	19101	903 Sit Ave	2014-07-04	young_professional	high
3	Claran	Harrison	(512) 598-1365	interdum.feugiat@aol.org	1998-04-29	United States	Houston	77001	736-2063 Tellus St.	2014-02-15	frequent_shopper	high
4	Howard	Stafford	1-411-740-3269	omare.egestas@icloud.edu	1989-02-18	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	2014-08-21	premium	high
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	1998-09-26	United States	Philadelphia	19101	341-2821 Ultrices Av.	2013-04-28	frequent_shopper	high
6	Joel	Tyson	(718) 288-8020	gravidat.nunc.sed@yahoo.ca	1989-10-15	United States	San Jose	95101	888-2799 Amet Street	2013-08-16	young_professional	medium
7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	1981-12-04	United States	Chicago	60601	8627 Malesuada Rd.	2013-01-15	frequent_shopper	high
8	Nasim	Franks	(692) 157-3469	egestas.aliquam.fringilla@google.ca	1993-08-01	United States	New York	10001	Ap #251-7144 Integer St.	2013-02-16	young_professional	medium
9	Mannix	Mocan	(590) 883-2184	aliquam.nisi@outlook.com	1987-01-24	United States	San Antonio	78201	647-3080 Lacus. St.	2014-02-15	young_professional	medium
10	Robert	McCarthy	(324) 746-6771	fermentum@protonmail.com	1984-04-30	United States	San Jose	95101	P.O. Box 773	2013-10-19	premium	high
11	Joan	Baird	(981) 429-8106	et.11@outlook.net	1990-02-25	United States	Los Angeles	90001	P.O. Box 687	2013-01-11	premium	very_high
12	Benedict	Wheeler	1-515-824-2855	tincidunt.donec.vitae@hotmail.co.uk	1999-08-06	United States	Phoenix	85001	748-8694 Porttitor Avenue	2013-05-14	student	medium
13	Allegro	Stanton	1-927-753-6488	proin.eget@protonmail.ca	1990-05-19	United States	New York	10001	4457 Ante. Av.	2013-10-29	frequent_shopper	medium
14	Sara	Flynn	1-311-646-9333	integer@outlook.net	1988-12-27	United States	Los Angeles	90001	P.O. Box 865	2014-09-08	young_professional	medium
15	Noelani	Patrick	1-723-488-5894	sem.magna@google.com	1993-09-17	United States	Los Angeles	90001	596-5044 Sapien	2014-09-15	premium	high
16	Eric	Roth	1-218-549-8253	lorem.st@yahoo.net	1988-09-07	United States	San Diego	92101	P.O. Box 341	2015-10-01	young_professional	medium

users 14 x

Output

#	Time	Action	Message	Duration / Fetch
41	21:06:14	CREATE INDEX idx_transactions_user_id ON transactions(user_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
42	21:06:17	SET GLOBAL local_infile = ON	0 row(s) affected	0.000 sec
43	21:06:20	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.000 sec / 0.000 sec
44	21:09:20	LOAD DATA LOCAL INFILE 'C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex 8__american_users.csv' INTO TABLE users FI...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.046 sec
45	21:09:25	LOAD DATA LOCAL INFILE 'C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex 8__european_users.csv' INTO TABLE users FI...	3390 row(s) affected Records: 3390 Deleted: 0 Skipped: 0 Warnings: 0	0.125 sec
46	21:09:28	SELECT * FROM users	5000 row(s) returned	0.000 sec / 0.015 sec

Activate Windows
Go to Settings to activate Windows.

Cargamos datos a la tabla de tarjetas:

```
-- [OP047]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-
Ex.8__credit_cards.csv"
INTO TABLE cards
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\\n'
IGNORE 1 ROWS
(@id, @user_id, @iban, @pan, @pin, @cvv, @track1, @track2,
 @expiring, @card_t, @renewal)
SET
  id = TRIM(@id),
  user_id = CAST(@user_id AS UNSIGNED),
  iban = TRIM(@iban),
  pan = REPLACE(@pan, ' ', ''),
  pin = TRIM(@pin),
  cvv = TRIM(@cvv),
  track1 = @track1,
  track2 = @track2,
  expiring_date = STR_TO_DATE(@expiring, '%m/%d/%Y'),
  card_type = TRIM(@card_t),
  renewal_flag = CAST(@renewal AS BINARY)
;
```

Comprobamos que los datos se cargaron:

```
-- [OP048]
SELECT * FROM cards;
```

Result Grid											
id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date	card_type	renewal_flag	
CS-4857	276	ES415679416676968711699208	2314242385113924	1819	467	%B2314242385113924*LWCBLUWCBUD^22...	%B2314242385113924*24101015183631649	2028-09-25	debit	1	
CS-4858	277	NL252053037609851262242244	6582720299715533	3964	817	%B6582720299715533*TTQWVTTQWV^2404...	%B6582720299715533*2411101045462727	2028-12-28	debit	0	
CS-4859	278	SE720084318584364771497262	8861684536289642	4983	277	%B8861684536289642*COFSGDCOFSGD^280...	%B8861684536289642*25021017616653717	2028-11-26	credit	0	
CS-4860	279	SE165574078045314365283427	2481155515498459	6876	661	%B2481155515498459*TIJUTIJUTIJU^31040...	%B2481155515498459*26021015144143957	2028-07-27	prepaid	0	
CS-4861	280	SE556762210431025196620841	1308930301149557	5710	398	%B1308930301149557*HPOBNZHPQBNZ^330...	%B1308930301149557*28051017513050287	2028-04-26	credit	0	
CS-4862	281	NL9493964297104396850171325	6715617009807829	4042	174	%B6715617009807829*LDWWTLDWWT^33...	%B6715617009807829*22101017023704287	2028-11-26	credit	0	
CS-4863	282	FR638211851201116231488674	3140878019451394	5959	449	%B3140878019451394*OXOXXOXOXX^230...	%B3140878019451394*32101011586485997	2028-12-29	debit	0	
CS-4864	283	SE455021683679162752728201	5793672133649114	8481	139	%B5793672133649114*NHVBVRNHVBVR^30...	%B5793672133649114*23061018063671017	2028-02-26	debit	0	
CS-4865	284	PL151333896127633364400219	510152687251312	7847	903	%B510152687251312*MOXOHMOXOH^330...	%B510152687251312*30101016648627107	2028-11-28	debit	0	
CS-4866	285	ES18590591352228333641620	8080768801072613	9271	961	%B8080768801072613*MEBGOZMEBGOZ^321...	%B8080768801072613*28101017158856957	2028-02-25	debit	1	
CS-4867	286	NL760008759553492236474321	7761849537661098	4820	862	%B7761849537661098*LDRSOLLDRSOL^2811...	%B7761849537661098*22061017104113717	2028-11-25	credit	1	
CS-4868	287	DE604023930989318335747030	4160419663151801	6860	549	%B4160419663151801*KQUCOBKQUCO^270...	%B4160419663151801*25081012785879277	2028-08-25	credit	1	
CS-4869	288	IT280405825351365860527091	5892930716233310	4775	448	%B5892930716233310*DNSCAVDNSCAV^250...	%B5892930716233310*24031012537581287	2028-09-25	premium...	1	
CS-4870	289	DE312678675102861012587987	2227240879956178	5872	715	%B2227240879956178*NNZJTRNNZJTR^3401...	%B2227240879956178*34111014300322227	2028-01-25	debit	1	
CS-4871	290	NL841882812537051501857688	6944143455556838	7992	643	%B6944143455556838*FRFELPRFELU^2707...	%B6944143455556838*32121013637162387	2028-07-26	credit	0	
CS-4872	291	PT694639039284374217174740	5035418997919459	3642	191	%B5035418997919459*ISXNYSXNYS^33080...	%B5035418997919459*28121017903140557	2028-12-26	credit	0	

Cargamos datos a la tabla de empresas:

```
-- [OP049]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-
Ex.8__companies.csv"
INTO TABLE companies
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@id, @name, @phone, @email, @country, @website,
 @category, @price)
SET
id = TRIM(@id),
name = TRIM(@name),
phone = TRIM(@phone),
email = TRIM(@email),
country = TRIM(@country),
website = TRIM(@website),
category = TRIM(@category),
price = TRIM(@price)
;
```

Comprobamos que los datos se cargaron:

```
-- [OP050]
SELECT * FROM companies;
```

id	name	phone	email	country	website	category	price
b-2222	Ac Fermentum Incorporated	+64 601 244 6320	donec.porrtitor.tellus@yahoo.net	Germany	https://www.ac-fermentum-incorporated.com	Beauty	mid-market
b-2226	Magna A Neque Industries	+33 823 560 3779	rius.donec.nibh@cloud.org	Australia	https://www.magna-a-neque-industries.com	Travel	budget
b-2230	Fusce Corp.	+29 657 646 4324	rius@protonmail.edu	United States	https://www.fusce-corp.com	Home	mid-market
b-2234	Convallis In Incorporated	+6 693 982 1255	mauris.ut@aol.co.uk	Germany	https://www.convallis-in-incorporated.com	Sports	mid-market
b-2238	Ante Iaculis Nec Foundation	+69 914 549 1009	sed.dctum.prim@outlook.ca	New Zealand	https://www.ante-iaculis-nec-foundation.com	Toys	mid-market
b-2242	Donec Ltd	+56 897 341 8418	at.iaculis@hotmail.co.uk	Norway	https://www.donec-ltd.com	Home	mid-market
b-2246	Sed Nunc Ltd	+5 218 900 3193	nibh@yahoo.org	United Kingdom	https://www.sed-nunc-ltd.com	Home	mid-market
b-2250	Amet Nulla Donec Corporation	+96 338 276 8449	mattis.integer.eu@protonmail.net	Italy	https://www.amet-nulla-donec-corporation.com	Gaming	budget
b-2254	Nascetur Ridiculus Mus Inc.	+63 316 953 7970	suspendisse.dui@icloud.net	United States	https://www.nascetur-ridiculus-mus-inc.com	Food	budget
b-2258	Vestibulum Lorem PC	+27 788 560 5000	aenean.massa.integer@aol.net	Belgium	https://www.vestibulum-lorem-pc.com	Electronics	premium
b-2262	Gravida Sagittis LLP	+98 204 198 9702	turpis.vitae@google.ca	Sweden	https://www.gravida-sagittis-llp.com	Sports	premium
b-2266	Mus Aenean Eget Foundation	+1 504 922 3000	mi.duis@hotmail.net	Sweden	https://www.mus-aenean-eget-foundation.com	Sports	mid-market
b-2270	Dis Parturient Institute	+69 962 744 6989	purus@protonmail.org	Ireland	https://www.ds-parturient-institute.com	Food	premium
b-2274	Sed LLC	+3 536 122 8140	at@outlook.com	Belgium	https://www.sed-llc.com	Fashion	mid-market
b-2278	Ardu LLP	+53 776 813 5178	dui@aol.ca	Norway	https://www.ardu-llp.com	Home	mid-market
b-2282	Pretium Neque Corp.	+17 184 177 7600	eleifend.nec.malesuada@proton...	Australia	https://www.pretium-neque-corp.com	Electronics	mid-market

Cargamos datos a la tabla de productos:

```
-- [OP051]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv"
INTO TABLE products
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@id, @name, @price, @color, @weight, @w_id, @category,
 @brand, @cost, @launch_date)
SET
  id          = CAST(@id AS UNSIGNED),
  name        = TRIM(@name),
  price       = CAST(REPLACE(@price, '$', '') AS DECIMAL(8,2)),
  hex_color   = UPPER(REPLACE(TRIM(@color), '#', '')),
  weight      = CAST(@weight AS DECIMAL(8,2)),
  warehouse_id = REPLACE(TRIM(@w_id), 'WH-', ''),
  category    = TRIM(@category),
  brand       = TRIM(@brand),
  cost        = CAST(REPLACE(@cost, '$', '') AS DECIMAL(8,2)),
  launch_date = STR_TO_DATE(@launch_date, '%Y-%m-%d')
;
```

Comprobamos que los datos se cargaron:

```
-- [OP052]
SELECT * FROM products;
```

Result Grid

Filter Rows

13

Wrap Cell Content: 13

#	id	name	price	hex_color	weight	warehouse_id	category	brand	cost	launch_date
1	Drevwolf Stannis	161.11	7C7C7C	1.00	4	Sports	Runway Pro	63.84	2014-11-01	
2	Tarly Stark	9.24	919191	2.00	3	Gaming	Questbyte	4.74	2013-02-07	
3	duel tourney Lannister	171.13	D8D8D8	1.50	2	Food	Daily Pantry	107.07	2013-08-18	
4	warden south duel	71.89	111111	3.00	1	Books	NorthPress	39.17	2013-12-31	
5	skywalker ewok	171.22	D8D8D8	3.20	0	Home	NordHaus	83.69	2014-07-17	
6	dooku solo	136.60	C4C4C4	0.80	1	Home	NordHaus	66.78	2014-03-14	
7	north of Casterly	63.33	878787	0.60	2	Electronics	Nexa	37.41	2013-10-10	
8	Winterfell	32.37	383838	1.40	3	Travel	Voyago	15.20	2013-01-27	
9	Winterfell	76.40	858585	1.20	4	Books	Campus Read	40.42	2013-08-14	
10	Karstark Dorne	119.52	F4F4F4	2.40	5	Food	FreshBox	89.07	2013-10-04	
11	Karstark Dorne	49.70	141414	2.70	6	Electronics	Nexa	34.45	2014-09-01	
12	duel Drevwolf	181.60	ABA8A8	2.10	7	Gaming	Questbyte	99.46	2013-10-11	
13	palpatine chewbacca	139.59	2B2B2B	1.00	8	Travel	CityHop	82.76	2014-04-26	
14	Drevwolf	147.53	C4C4C4	2.00	9	Home	NordHaus	71.07	2014-03-29	
15	Stannis warden	194.29	D8D8D8	1.50	10	Home	CasaNova	107.93	2014-10-21	
16	the duel warden	180.91	666666	3.00	11	Home	NordHaus	101.11	2014-05-31	

products 17 x

Result Grid

Form Editor

Field Types

Query Stats

ApplyRevertContext HelpSnippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
47	21:10:06	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__credit_cards.csv" INTO TABLE cards FIEL...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.188 sec
48	21:10:10	SELECT * FROM cards	5000 row(s) returned	0.000 sec / 0.016 sec
49	21:10:44	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__companies.csv" INTO TABLE companies F...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.000 sec
50	21:10:47	SELECT * FROM companies	100 row(s) returned	0.000 sec / 0.000 sec
51	21:11:10	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv" INTO TABLE products FIEL...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
52	21:11:13	SELECT * FROM products	100 row(s) returned	0.000 sec / 0.000 sec

Activate Windows
Go to Settings to activate Windows.

Cargamos datos a la tabla de transacciones:

```
-- [OP053]
LOAD DATA LOCAL
INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-
Ex.8__transactions.csv"
INTO TABLE transactions
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\\n'
IGNORE 1 ROWS
(@id, @card_id, @com_id, @instant, @amount, @declined,
 @products, @user_id, @lat, @lon, @discount, @tax, @shipping,
 @channel, @campaign, @device, @international, @decl_reason,
 @distance)
SET
id = TRIM(@id), -- VARCHAR(40)
card_id = TRIM(@card_id), -- VARCHAR(15)
company_id = TRIM(@com_id), -- VARCHAR(15)
instant = STR_TO_DATE(@instant, '%Y-%m-%d %H:%i:%s'), -- TIMESTAMP
amount = CAST(@amount AS DECIMAL(8,2)), -- DECIMAL(8,2)
declined = CAST(@declined AS BINARY), -- BOOL
product_ids = TRIM(@products), -- VARCHAR(255)
user_id = CAST(@user_id AS UNSIGNED), -- INT
latitude = CAST(@lat AS FLOAT), -- FLOAT
longitude = CAST(@lon AS FLOAT), -- FLOAT
discount = CAST(@discount AS DECIMAL(8,2)), -- DECIMAL(8,2)
tax = CAST(@tax AS DECIMAL(8,2)), -- DECIMAL(8,2)
shipping = CAST(@shipping AS DECIMAL(8,2)), -- DECIMAL(8,2)
channel = TRIM(@channel), -- VARCHAR(20)
campaign_id = TRIM(@campaign), -- VARCHAR(30)
device_type = TRIM(@device), -- VARCHAR(20)
international = CAST(@international AS BINARY), -- BOOL
decline_reason = TRIM(@decl_reason), -- VARCHAR(255)
```

```
distance_km = CAST(@distance AS DECIMAL(8,2)) -- DECIMAL(8,2);
```

Comprobamos que los datos se cargaron:

```
-- [OP054]
SELECT * FROM transactions;
```

The screenshot shows a database management tool interface. The top part displays a table with columns: id, card_id, company_id, instant, amount, declined, product_ids, user_id, latitude, longitude, discount, tax, shipping, channel, campaign_id, and device_type. The table contains 18 rows of transaction data. The bottom part shows a log of actions performed, including loading data from CSV files into tables and executing SQL queries. The log indicates that 100,000 rows were affected in the transactions table.

id	card_id	company_id	instant	amount	declined	product_ids	user_id	latitude	longitude	discount	tax	shipping	channel	campaign_id	device_type
00043A49-2949-494B-A5DD-A5BAE3B8190D	CcS-9294	b-2458	2024-08-28 07:16:46	433.51	0	16, 26, 97, 87	4713	46.1999	1.43554	34.17	72.25	0.00	online	SUMMER_TRAVEL	mobile
000447FE-8650-40CF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972	12.2218	19.92	29.86	6.99	online	CHRISTMAS_PEAK	tablet
00045D68-ED2E-4F2F-8186-CE074D875D0D	CcS-6699	b-2390	2020-07-14 15:37:45	332.38	0	30, 11, 16, 81	2118	29.7573	-95.3796	16.10	22.47	0.00	store	SUMMER_TRAVEL	pos
000481C3-1C26-4FEF-8340-4C0EB004EB8D	CcS-6696	b-2230	2017-09-04 19:44:53	175.00	0	72	2115	53.5489	-113.503	1.05	14.45	0.00	store	NO_CAMPAIGN	pos
00051A44-9CDE-4268-B070-C306A2A1E2E2	CcS-7606	b-2266	2017-01-05 18:19:25	158.55	0	18	3025	52.2084	5.69081	25.31	25.96	8.99	marketplace	WINTER_SALE	desktop
0008A312-EDFE-4A4F-6C99-E9C3E2C3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	317.97	0	35, 33, 19	215	53.5535	-113.499	2.87	26.25	0.00	store	NO_CAMPAIGN	pos
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2023-12-31 00:06:36	457.51	0	93, 55, 28, 91	2928	52.5225	13.5529	5.52	79.40	0.00	online	NO_CAMPAIGN	mobile
0009D494-6245-4DF9-955D-2C08A191CFFB	CcS-8483	b-2526	2017-07-18 07:52:02	225.49	0	55, 8, 72	3902	45.492	-73.5706	2.85	17.55	12.99	online	NO_CAMPAIGN	tablet
000A1DEC-DCB6-4AB2-A619-7DA8B04A262	CcS-6467	b-2558	2018-09-08 05:29:58	374.41	0	46, 56, 73	1886	55.7425	-3.30009	27.93	62.40	0.00	store	BACK_TO_SCHOOL	pos
000A1E64-1414-40B0-9D92-5678A4D958E2	CcS-5966	b-2550	2022-09-17 04:02:19	378.00	0	6, 89, 19	1385	52.0821	5.28424	57.31	65.60	0.00	online	BACK_TO_SCHOOL	desktop
000A5879-3472-41D9-AF60-42D35038543C	CcU-4569	b-2590	2020-02-07 23:03:45	185.46	0	58, 61, 55	42	39.949	-75.1719	1.62	11.66	12.99	online	NO_CAMPAIGN	mobile
000AEDD4-F06E-4146-804A-5A2FC73110D7	CcS-8134	b-2426	2017-07-10 07:11:41	187.69	0	62	3553	49.3748	-123.267	20.41	15.17	3.99	online	SUMMER_TRAVEL	mobile
000BCFB8-A4AB-4E8E-A148-2E7A0E1B385	CcS-6020	b-2598	2019-05-28 04:23:02	108.70	0	36	1439	33.4472	-112.072	1.39	6.88	6.99	marketplace	NO_CAMPAIGN	desktop
000B0852-E25B-4785-93D9-27051425D45B	CcS-5445	b-2330	2024-07-31 09:45:39	117.35	0	8, 91	864	51.4317	5.47489	0.92	19.15	6.99	online	NO_CAMPAIGN	mobile
000DAABF-ABEA-4F32-A090-85CC07E5D9E9	CcS-6404	b-2454	2023-07-26 22:39:50	345.24	0	76, 19, 89	1823	41.7392	12.6293	15.20	62.26	0.00	store	SUMMER_TRAVEL	pos

transactions 18 x

Output

#	Time	Action	Message	Duration / Fetch
49	21:10:44	LOAD DATA LOCAL INFILE 'C:\Program Files\MySQL\MySQL Server 8.0\Uploads\N1-Ex.8_companies.csv' INTO TABLE companies F...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.000 sec
50	21:10:47	SELECT * FROM companies	100 row(s) returned	0.000 sec / 0.000 sec
51	21:11:10	LOAD DATA LOCAL INFILE 'C:\Program Files\MySQL\MySQL Server 8.0\Uploads\N1-Ex.8_products.csv' INTO TABLE products FIEL...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
52	21:11:13	SELECT * FROM products	100 row(s) returned	0.000 sec / 0.000 sec
53	21:11:35	LOAD DATA LOCAL INFILE 'C:\Program Files\MySQL\MySQL Server 8.0\Uploads\N1-Ex.8_transactions.csv' INTO TABLE transaction...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	6.625 sec
54	21:11:48	SELECT * FROM transactions	100000 row(s) returned	0.000 sec / 0.328 sec

Ex. 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
-- [OP055]
WITH
  tr_count AS (
    SELECT
      t.user_id,
      COUNT(1) AS n
    FROM transactions AS t
    GROUP BY t.user_id
    HAVING n > 80
  )
SELECT u.*, c.n
FROM users AS u
JOIN tr_count AS c
ON u.id = c.user_id;
```

Result Grid												
Filter Rows:												
id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	segment	income_band
185	Molly	Gillam	0800 120 8023	donec@outlook.co.uk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 MI Rd.	2013-08-11	frequent_shopper	high
289	Dwight	Hwrcu	+98-309-8797	dwg.hwrcu@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwrcu Street	2013-12-11	occasional	low
318	Bryr	Astuw	+33-120-9644	brnr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street	2013-05-29	frequent_shopper	very_high
454	Sfzsh	Xgvfrdxs	+58-495-3945	sfzsh.xgvfrdxs@example.com	1962-08-28	Poland	Gdansk	80-001	52 Xgvfrdxs Street	2014-08-30	established	medium

#	Time	Action	Message	Duration / Fetch
50	21:10:47	SELECT * FROM companies	100 row(s) returned	0.000 sec / 0.000 sec
51	21:11:10	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex 8_products.csv" INTO TABLE products FIEL...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
52	21:11:13	SELECT * FROM products	100 row(s) returned	0.000 sec / 0.000 sec
53	21:11:35	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex 8_transactions.csv" INTO TABLE transaction...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	6.625 sec
54	21:11:48	SELECT * FROM transactions	100000 row(s) returned	0.000 sec / 0.328 sec
55	21:12:45	WITH tr_count AS (SELECT t.user_id, COUNT(1) AS n FROM transactions AS t GROUP BY t.user_id HAVING n > 80) SE...	4 row(s) returned	0.031 sec / 0.000 sec

Ex. 10

Mostra la mitjana d' amount per IBAN de les targetes de crèdit a la companyia Donec Ltd , utilitza almenys 2 taules.

```
-- [OP056]
SELECT
    c.iban,
    AVG(t.amount) AS mitjana
FROM transactions AS t
JOIN cards AS c
ON t.card_id = c.id
JOIN companies AS cm
ON t.company_id = cm.id
WHERE cm.name = "Donec Ltd"
GROUP BY c.iban;
```

Result Grid												
Filter Rows:												
iban	mitjana											
CA901355773586771626108990	361.010000											
US729212642816955651428791	142.560000											
US899450803230933276819251	246.900000											
US581097432688325541142297	154.010000											
CA441226267105857473570574	250.510000											
CA332171962352073015086775	203.950000											
US985359628701198880628720	438.160000											
US788111972997693675524914	520.366667											
CA867886529258577442049898	204.630000											
US450950742400388023162002	165.075000											

#	Time	Action	Message	Duration / Fetch
53	21:11:35	LOAD DATA LOCAL INFILE "C:\\Program Files\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex 8_transactions.csv" INTO TABLE transaction...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	6.625 sec
54	21:11:48	SELECT * FROM transactions	100000 row(s) returned	0.000 sec / 0.328 sec
55	21:12:45	WITH tr_count AS (SELECT t.user_id, COUNT(1) AS n FROM transactions AS t GROUP BY t.user_id HAVING n > 80) SE...	4 row(s) returned	0.031 sec / 0.000 sec
56	21:13:33	SELECT c.iban, AVG(t.amount) AS mitjana FROM transactions AS t JOIN cards AS c ON t.card_id = c.id JOIN companies AS cm ON t.compa...	371 row(s) returned	0.015 sec / 0.000 sec

Lvl. 2

Ex. 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

Identifica los cinco días en los cuales se generó la mayor cantidad de ingresos en la empresa.

Se refiere a una empresa concreta en singular:

- No se nos aclara cuál es la empresa en concreto.
- No se nos pide que hagamos una suma de ingresos.
- No se nos pide un top 5 entre todas.
- No se nos pide un top 5 para cada empresa.

Por contexto, deduzco que "la empresa", se refiere específicamente a `Donec Ltd`.

Muestra la data de cada transacción juntamente con el ingreso total (del día).

"Cada transacción" sugiere que entreguemos una data (i.e. día) individual para cada transacción (presupongo que delimitándonos a los cinco días que hemos identificado).

- Habrá 5 datas distintas entre más de 5 transacciones que hayan ocurrido en los días identificados.
- No se nos pide que mostremos transacciones individuales explícitamente.

Por contexto asumo que el enunciado está mal expresó y realmente se quería decir:

Muestra la data de los cinco días que has identificado y el ingreso total de cada uno de estos cinco días.

Interpretación del enunciado:

Identifica los cinco días en los cuales se generó la mayor cantidad de ingresos en la empresa `Donec Ltd`. Muestra la data de los cinco días que has identificado y el ingreso total de `Donec Ltd` en cada uno de estos cinco días.

```
-- [OP057]
WITH
  -- aggregate all "Donec Ltd" transaction amounts by date.
  aggregated AS (
    SELECT
      DATE(t.instant) AS t_day,
      SUM(t.amount) AS total
    FROM transactions AS t
    JOIN companies AS c
    ON t.company_id = c.id
    WHERE c.name = "Donec Ltd"
    GROUP BY t_day
```

```

),
-- Rank the dates by total:
-- top 1, 2, 3, 4, 5 <- no ties
-- top 1, 2, 2, 4, 5 <- tie second place
-- top 1, 1, 1, 1, 5 <- four way tie first place
ranked AS (
    SELECT
        a.*,
        RANK() OVER(ORDER BY a.total DESC) AS rn
    FROM aggregated AS a
    ORDER BY a.total DESC
)
SELECT *
FROM ranked AS r
WHERE r.rn <= 5;

```

Result Grid				Filter Rows:	Export:	Wrap Cell Contents:
t_day	total	rn				
2022-12-13	1561.10	1				
2018-11-26	1231.38	2				
2015-05-09	1208.56	3				
2024-10-25	974.56	4				
2015-12-04	974.22	5				

#	Time	Action	Message	Duration / Fetch
54	21:11:48	SELECT * FROM transactions	100000 row(s) returned	0.000 sec / 0.328 sec
55	21:12:45	WITH tr_count AS (SELECT t.user_id, COUNT(1) AS n FROM transactions AS t GROUP BY t.user_id HAVING n > 80) SE...	4 row(s) returned	0.031 sec / 0.000 sec
56	21:13:33	SELECT c.iban, AVG(t.amount) AS mitjana FROM transactions AS t JOIN cards AS c ON t.card_id = c.id JOIN companies AS cm ON t.compa...	371 row(s) returned	0.015 sec / 0.000 sec
57	21:14:23	WITH --aggregate all "Donec Ltd" transaction amounts by date. aggregated AS (SELECT DATE#instant) AS t_day, SUM(t.amount) ...	5 row(s) returned	0.000 sec / 0.000 sec

Ex. 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018, 13 de març del 2024.

Ordena els resultats de major a menor quantitat.

```

-- [OP058]
SELECT
    c.name,
    c.phone,
    c.country,
    DATE(t.instant) AS t_day,
    t.amount
FROM companies AS c
JOIN transactions AS t
ON t.company_id = c.id
WHERE DATE(t.instant) in (
    '2015-4-29',

```

```
'2018-7-20',
'2024-3-13')
AND t.amount BETWEEN 350 AND 400
ORDER BY t.amount DESC;
```

name	phone	country	t_day	amount
Non Magna LLC	+94 833 862 7544	United Kingdom	2015-03-29	398.89
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Quam A Fels Industries	+14 677 174 5530	Italy	2015-03-29	378.65
Pede Cum Ltd	+26 625 796 6126	Norway	2018-07-20	358.64
Netus El Molestuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

#	Time	Action	Message	Duration / Fetch
55	21:12:45	WITH tr_count AS (SELECT t.user_id, COUNT(1) AS n FROM transactions AS t GROUP BY t.user_id HAVING n > 80) SE...	4 row(s) returned	0.031 sec / 0.000 sec
56	21:13:33	SELECT c.id, AVG(t.amount) AS mitjana FROM transactions AS t JOIN cards AS c ON t.card_id = c.id JOIN companies AS cm ON t.compa...	371 row(s) returned	0.015 sec / 0.000 sec
57	21:14:23	WITH -- aggregate all "Donec Ltd" transaction amounts by date. agregated AS (SELECT DATE(t.instant) AS t_day, SUM(t.amount) ...	5 row(s) returned	0.000 sec / 0.000 sec
58	21:14:59	SELECT c.name, c.phone, c.country, DATE(t.instant) AS t_day, t.amount FROM companies AS c JOIN transactions AS t ON t.company_j...	6 row(s) returned	0.390 sec / 0.000 sec

Ex. 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```
-- [OP059]
SELECT
  t.id,
  t.name,
  CASE
    WHEN n_tr > 400 THEN 'mas de 400'
    WHEN n_tr = 400 THEN '400'
    ELSE 'menos de 400'
  END AS category
FROM (
  SELECT
    c.id,
    c.name,
    COUNT(t.id) AS n_tr
  FROM companies AS c
  JOIN transactions AS t
  ON t.company_id = c.id
  GROUP BY c.id
  ORDER BY n_tr DESC
) AS t;
```

Result Grid			Filter Rows	Export	Wrap Cell Contents	
id	name	category				
b-2222	Ac Fermentum Incorporated	mas de 400				
b-2302	Nunc Interdum Incorporated	mas de 400				
b-2330	Donec Fringilla PC	mas de 400				
b-2366	Mauris Institute	mas de 400				
b-2614	Rutrum Non Inc.	mas de 400				
b-2350	Aliquet Vel Vulputate Incorporated	mas de 400				
b-2556	Aliquam PC	mas de 400				
b-2574	Orci Adipiscing Limited	mas de 400				
b-2478	Etiam Bibendum Fermentum Industries	mas de 400				
b-2550	Auctor Mauris Corp.	mas de 400				
b-2266	Mus Aenean Eget Foundation	mas de 400				
b-2326	Enim Conditum Ltd	mas de 400				

#	Time	Action	Message	Duration / Fetch
56	21:13:33	SELECT c.id, AVG(t.amount) AS mjtiana FROM transactions AS t JOIN cards AS c ON t.card_id = c.id JOIN companies AS cm ON t.compa...	371 row(s) returned	0.015 sec / 0.000 sec
57	21:14:21	WITH -- aggregate all "Donec Ltd" transaction amounts by date. aggregated AS (SELECT DATE(t.instant) AS t_day, SUM(t.amount) ...	5 row(s) returned	0.000 sec / 0.000 sec
58	21:14:59	SELECT c.name, c.phone, c.country, DATE(t.instant) AS t_day, t.amount FROM companies AS c JOIN transactions AS t ON t.company_j...	6 row(s) returned	0.390 sec / 0.000 sec
59	21:15:53	SELECT t.id, t.name, CASE WHEN n_tr > 400 THEN 'mas de 400' WHEN n_tr = 400 THEN '400' ELSE 'menos de 400' END AS ca...	100 row(s) returned	0.078 sec / 0.000 sec

Ex. 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Observamos antes de la operación:

```
-- [OP060]
SELECT *
FROM transactions AS t
WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid

Filter Rows

Edit

Export/Import

Wrap Cell Contents

id	card_id	company_id	instant	amount	declined	product_ids	user_id	latitude	longitude	discount	tax	shipping	channel	campaign_id	device_type	international
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	C6S-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972	12.2218	19.92	29.86	6.99	online	CHRISTMAS_PEAK	tablet	0

transactions 24

Apply Revert

Context Help Snippets

Action Output

#	Time	Action	Message	Duration / Fetch
57	21:14:23	WITH -- aggregate all "Donec Ltd" transaction amounts by date. aggregated AS (SELECT DATE(t.instant) AS t_day, SUM(t.amount) ...	5 row(s) returned	0.000 sec / 0.000 sec
58	21:14:59	SELECT c.name, c.phone, c.country, DATE(t.instant) AS t_day, t.amount FROM companies AS c JOIN transactions AS t ON t.company_j...	6 row(s) returned	0.390 sec / 0.000 sec
59	21:15:53	SELECT t.id, t.name, CASE WHEN n_tr > 400 THEN 'mas de 400' WHEN n_tr = 400 THEN '400' ELSE 'menos de 400' END AS ca...	100 row(s) returned	0.078 sec / 0.000 sec
60	21:16:29	SELECT * FROM transactions AS t WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.000 sec / 0.000 sec

Eliminamos la transacción:

```
-- [OP061]
DELETE FROM transactions AS t
WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Observamos que ha sido eliminada:

```
-- [OP062]
SELECT *
FROM transactions AS t
WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid

Filter Rows

idcard_idcompany_idinstantamountdeclinedproduct_iduser_idlatitudelongitudediscounttaxshippingchannelcampaign_iddevice_typeinternationaldecline_reasondistance_km

transactions 25 x

ApplyRevertContext HelpSnippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
59	21:15:53	SELECT t.id, t.name, CASE WHEN n_r > 400 THEN 'mas de 400' WHEN n_r = 400 THEN '400' ELSE 'menos de 400' END AS ca...	100 row(s) returned	0.078 sec / 0.000 sec
60	21:16:29	SELECT * FROM transactions AS t WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned	0.000 sec / 0.000 sec
61	21:17:03	DELETE FROM transactions AS t WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.016 sec
62	21:17:07	SELECT * FROM transactions AS t WHERE t.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec

Ex. 5

La secció de màrqueting desitja tenir accés a informació específica per a fer anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada `VistaMarketing` que contingui la següent informació:

- Nom de la companyia.
- Telèfon de contacte.
- País de residència.
- Mitjana de compra realitzat per cada companyia.

Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

Asumo que una venta es una transacción no denegada.

```
-- [OP063]
CREATE OR REPLACE VIEW VistaMarketing AS
SELECT
    c.name, -- nom
    c.phone, -- telefon
    c.country, -- pais residencia
    AVG(t.amount) AS avg_sale -- mitjana de compra
FROM companies AS c
JOIN transactions AS t
ON t.company_id = c.id
WHERE t.declined = FALSE
GROUP BY
    c.name,
    c.phone,
    c.country
ORDER BY avg_sale DESC;
```

```
-- [OP064] Observamos la nueva vista.
SELECT *
```

```
FROM vistamarketing;
```

The screenshot shows a database management tool interface. At the top, a table named 'vistamarketing' is displayed with columns: name, phone, country, and avg_sale. The table contains 20 rows of data. Below the table, the 'Output' pane shows the execution of a SQL script. The script consists of four lines: a DELETE statement, a SELECT statement, a CREATE OR REPLACE VIEW statement, and another SELECT statement. The output shows the execution time and the number of rows affected or returned for each statement.

#	Time	Action	Message	Duration / Fetch
61	21:17:03	DELETE FROM transactions AS t WHERE t.id = '000447FE-8650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	0.016 sec
62	21:17:07	SELECT * FROM transactions AS t WHERE t.id = '000447FE-8650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec
63	21:18:16	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.name, -- nom c.phone, -- telefon c.country, -- pais residencia AVG(t.amount) A...	0 row(s) affected	0.016 sec
64	21:18:20	SELECT * FROM vistamarketing	100 row(s) returned	0.438 sec / 0.000 sec

Lvl. 3

Ex. 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu.

Partint d'aquesta taula respon: Quantes targetes estan actives?

```
-- [OP065]
DROP TABLE IF EXISTS cards_activity;
```

```
-- [OP066]
CREATE TABLE IF NOT EXISTS cards_activity AS
WITH
  ranked AS (
    SELECT
      c.id,
      t.declined,
      ROW_NUMBER() OVER(
        PARTITION BY c.id
        ORDER BY t.instant DESC
      ) AS rn
    FROM transactions AS t
    JOIN cards AS c
    ON t.card_id = c.id
  ),
  activity AS (
    SELECT
      r.id,
```

```

        SUM(r.declined) AS decl_count
    FROM ranked AS r
    WHERE r.rn < 4
    GROUP BY r.id
)
SELECT
    a.id,
    CASE
        WHEN a.decl_count > 2 THEN FALSE
        ELSE TRUE
    END AS is_active
FROM activity AS a;

```

Observamos la tabla:

```

-- [OP067]
SELECT * FROM cards_activity;

```

The screenshot shows the SQL Server Enterprise Manager interface. The 'Result Grid' displays the 'cards_activity' table with columns 'id' and 'is_active'. The 'Output' window shows the execution logs for the query 'SELECT * FROM cards_activity'. The logs indicate that 5000 rows were returned.

#	Time	Action	Message	Duration / Fetch
64	21:18:20	SELECT * FROM vistamarketing	100 row(s) returned	0.438 sec / 0.000 sec
65	21:19:09	DROP TABLE IF EXISTS cards_activity	0 row(s) affected. 1 warning(s): 1051 Unknown table '21e8.cards_activity'	0.015 sec
66	21:19:12	CREATE TABLE IF NOT EXISTS cards_activity AS WITH ranked AS (SELECT c.id, t.declined, ROW_NUMBER() OVER(...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.797 sec
67	21:19:19	SELECT * FROM cards_activity	5000 row(s) returned	0.000 sec / 0.000 sec

¿Cuántas tarjetas son activas?

```

-- [OP068]
SELECT COUNT(1) AS n_active
FROM cards_activity AS c
WHERE c.is_active = TRUE;

```

The screenshot shows the SQL Server Enterprise Manager interface. The 'Result Grid' displays the 'n_active' table with a single row containing the value '5000'. The 'Output' window shows the execution logs for the query 'SELECT COUNT(1) AS n_active FROM cards_activity WHERE c.is_active = TRUE'. The logs indicate that 1 row was returned.

#	Time	Action	Message	Duration / Fetch
65	21:19:09	DROP TABLE IF EXISTS cards_activity	0 row(s) affected. 1 warning(s): 1051 Unknown table '21e8.cards_activity'	0.015 sec
66	21:19:12	CREATE TABLE IF NOT EXISTS cards_activity AS WITH ranked AS (SELECT c.id, t.declined, ROW_NUMBER() OVER(...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.797 sec
67	21:19:19	SELECT * FROM cards_activity	5000 row(s) returned	0.000 sec / 0.000 sec
68	21:19:51	SELECT COUNT(1) AS n_active FROM cards_activity AS c WHERE c.is_active = TRUE	1 row(s) returned	0.016 sec / 0.000 sec

Ex. 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de `products.csv` amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de `transaction` tens `product_ids`. Genera la següent consulta:

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Comento que los datos de producto de `products.csv` se han cargado en `Lvl.1 Ex.8`

```
-- [OP069]
DROP TABLE IF EXISTS orders;
```

```
-- [OP070]
CREATE TABLE IF NOT EXISTS orders (
  transaction_id VARCHAR(40),
  product_id     INT,
  PRIMARY KEY (transaction_id, product_id),
  FOREIGN KEY (transaction_id) REFERENCES transactions(id),
  FOREIGN KEY (product_id) REFERENCES products(id),
  INDEX idx_product_id (product_id),
  INDEX idx_transaction_id (transaction_id)
);
```

Para este ejercicio he tenido que consultar recursos externos para obtener inspiración. Con tal de demostrar que entiendo lo que he hecho:

Originalmente, cree esta comanda para solucionar el ejercicio. Es una simple función recursiva que descompone el field `t.product_ids` en una lista de `string`s, cada uno conteniendo el número de producto, a continuación genera una fila por cada id de producto y traduce el `string` a un número.

```
INSERT INTO orders
(transaction_id, product_id)
WITH
  RECURSIVE split AS (
    SELECT
      t.id AS tr,
      SUBSTRING_INDEX(t.product_ids, ',', 1) AS token,
      CASE
        WHEN t.product_ids LIKE '%,%'
        THEN SUBSTRING(t.product_ids, INSTR(t.product_ids, ',') + 1)
        ELSE NULL
      END AS remainder
    FROM transactions AS t
    WHERE
```

```

        t.product_ids IS NOT NULL
        AND t.product_ids <> ''
    UNION ALL
    SELECT
        s.tr,
        SUBSTRING_INDEX(s.remainder, ',', 1),
        CASE
            WHEN s.remainder LIKE '%,%'
            THEN SUBSTRING(s.remainder, INSTR(s.remainder, ',') + 1)
            ELSE NULL
        END
    FROM split AS s
    WHERE s.remainder IS NOT NULL
)
SELECT
    s.tr,
    CAST(s.token AS UNSIGNED)
FROM split AS s;

```

El primer `SELECT` (i.e. antes de `UNION ALL`) separa el primer token de la lista `t.product_ids`: para eso genera dos `string` `s`, `token` y `remainder`:

- `SUBSTRING_INDEX` Permite obtener el token, ya que puede recoger letras hasta la primera coma.
- `SUBSTRING` Permite recoger letras después de un índice especificado.
- `INSTR` Permite obtener el índice de la primera coma.

Un índice es el número que identifica la posición de una un objeto en un `array`, en este caso, el objeto es una letra, y el `array` es un `string`. Un `array` es una secuencia de elementos del mismo tipo almacenada en memoria de forma contigua (según las direcciones virtuales de memoria concedidas al programa)

El segundo `SELECT` repite el proceso, actuando igual que el primer `SELECT`, pero en vez de procesar `t.product_ids`, ingiere el `remainder` de su padre recursivo. Se ejecuta recursivamente (i.e. en este caso se podría decir, en un bucle) hasta que `remainder IS NULL`. No creo que haga falta aclararme más.

En el P2P se me mostró una forma más corta de hacerlo que aprovecha funcionalidades existentes, facilitando el proceso, muy probablemente haciendo el programa más eficiente, y evitando los bugs que pueda yo haber introducido al escribir esto a mano.

A diferencia de mi versión anterior, esta solución es simple:

1. Cogemos `product_ids` y le añadimos corchetes para formar un `JSON array` válido.
2. Usando `JSON query` seleccionamos todos los elementos en el `array`.

3. Cada elemento lo convertimos en una fila:

1. Cada `string` se convierte a un `INT`.

4. El `JOIN` con la `JSON_TABLE` es dependiente en elementos se `transactions`, y por defecto se trata como un `JOIN LATERAL`.

```
-- [OP071]
INSERT INTO orders
(transaction_id, product_id)
SELECT
    t.id,
    p.id
FROM transactions AS t
JOIN JSON_TABLE (
    CONCAT('[', t.product_ids, ']'),
    '$[*]' COLUMNS (id INT PATH '$')
) AS p;
```

Observamos que la tabla se rellenó correctamente:

```
-- [OP072]
SELECT *
FROM orders AS o;
```

The screenshot shows a database client interface. At the top, there's a 'Result Grid' tab with a table named 'orders'. The table has two columns: 'transaction_id' and 'product_id'. It contains 29 rows of data. Below the table, there's an 'Output' tab showing the execution log of SQL queries. The log shows the following queries and their results:

#	Time	Action	Message	Duration / Fetch
69	21:20:48	DROP TABLE IF EXISTS orders	0 row(s) affected. 1 warning(s): 1051 Unknown table '21e8.orders'	0.000 sec
70	21:20:51	CREATE TABLE IF NOT EXISTS orders (transaction_id VARCHAR(40), product_id INT, PRIMARY KEY (transaction_id, product_id), F...	0 row(s) affected	0.046 sec
71	21:20:58	INSERT INTO orders (transaction_id, product_id) SELECT t.id, p.id FROM transactions AS t JOIN JSON_TABLE (CONCAT(T, t.product_ids...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	7.594 sec
72	21:21:29	SELECT * FROM orders AS o	253388 row(s) returned	0.000 sec / 0.094 sec

¿Cuántas veces se ha vendido cada producto?

```
-- [OP073]
SELECT
    o.product_id,
    p.name,
    COUNT(o.transaction_id) AS n_sales
FROM orders AS o
JOIN transactions AS t
```

```

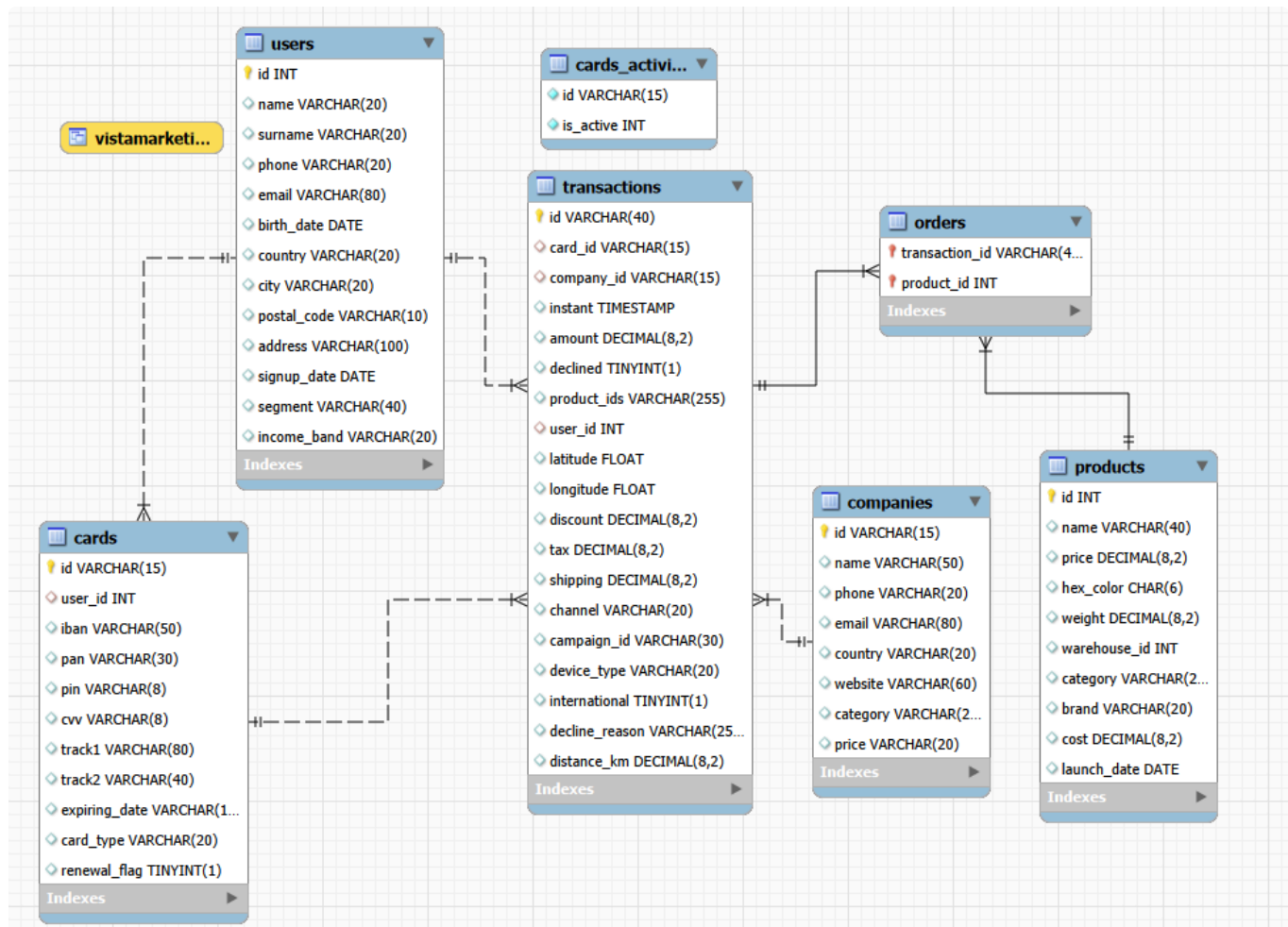
ON o.transaction_id = t.id
JOIN products AS p
ON o.product_id = p.id
WHERE t.declined = FALSE
GROUP BY o.product_id
ORDER BY o.product_id;

```

product_id	name	n_sales
1	Direwolf Stannis	2450
2	Tarly Stark	2552
3	duel tourney Lannister	2507
4	warden south duel	2563
5	skywalker evok	2530
6	dooku solo	2460
7	north of Casterly	2535
8	Winterfell	2473
9	Winterfell	2497
10	Karstark Dorne	2554
11	Karstark Dorne	2556
12	duel Direwolf	2531
13	palpatine chevbacca	2538
14	Direwolf	2470
15	Stannis warden	2505
16	the duel warden	2583
17	skywalker evok sith	2496

#	Time	Action	Message	Duration / Fetch
70	21:20:51	CREATE TABLE IF NOT EXISTS orders (transaction_id VARCHAR(40), product_id INT, PRIMARY KEY (transaction_id, product_id), F...	0 row(s) affected	0.046 sec
71	21:20:58	INSERT INTO orders (transaction_id, product_id) SELECT t.id, p.id FROM transactions AS t JOIN JSON_TABLE (CONCAT(T...t.product_ids...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	7.594 sec
72	21:21:29	SELECT * FROM orders AS o	253388 row(s) returned	0.000 sec / 0.094 sec
73	21:22:23	SELECT o.product_id, p.name, COUNT(o.transaction_id) AS n_sales FROM orders AS o JOIN transactions AS t ON o.transaction_id = t.id J...	100 row(s) returned	1.062 sec / 0.000 sec

Diagrama final



Es un modelo copo de nieve, pero si hubiera que designar una tabla central de hechos, esa sería la tabla de transacciones. Las demás tablas tienen función más dimensional, ya que llevan información contextual que enriquece a las transacciones. La tabla `orders` podría considerarse algo como una tabla de hechos... pero es más una extensión de la tabla de transacciones.

Relaciones

```
cards          » 1 <--> 0..N « transactions
cards          » 0..N <--> 1 « users
companies      » 1 <--> 0..N « transactions
orders         » 0..N <--> 1 « products
orders         » 0?..N <-> 1 « transactions
transactions   » 0..N <--> 1 « users
```